

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
TRUNG TÂM TIN HỌC

—oOo—

BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

Chương trình Lập trình viên Full-stack JavaScript — Khóa 312

LEARNQUIZ

NỀN TẢNG HỌC TẬP VÀ QUIZ TRỰC TUYẾN

Học viên thực hiện: **VŨ TÂM THIỆN TÍN**

Mã học viên:

Giảng viên hướng dẫn: **NGUYỄN LÊ HOÀNG THÔNG**

THÀNH PHỐ HỒ CHÍ MINH — THÁNG 8 NĂM 2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
TRUNG TÂM TIN HỌC

—oOo—

BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

Chương trình Lập trình viên Full-stack JavaScript — Khóa 312

LEARNQUIZ

NỀN TẢNG HỌC TẬP VÀ QUIZ TRỰC TUYẾN

Đề tài số 4

Giảng viên hướng dẫn: **NGUYỄN LÊ HOÀNG THÔNG**

Học viên thực hiện: **VŨ TÂM THIỆN TÍN**

Mã học viên:

Lớp: Full-stack JavaScript — Khóa 312

THÀNH PHỐ HỒ CHÍ MINH — THÁNG 8 NĂM 2026

LỜI CẢM ƠN

Đồ án này khép lại chương trình Lập trình viên Full-stack JavaScript — Khóa 312 tại Trung Tâm Tin Học, Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh.

Em xin chân thành cảm ơn quý thầy cô của Trung Tâm Tin Học đã truyền đạt kiến thức nền tảng và kinh nghiệm thực tiễn trong suốt bốn học phần của chương trình. Những buổi học về kiến trúc web, cơ sở dữ liệu quan hệ, thiết kế API và triển khai hệ thống là chất liệu trực tiếp làm nên sản phẩm được trình bày trong báo cáo này.

Em xin gửi lời cảm ơn sâu sắc tới thầy **Nguyễn Lê Hoàng Thông** — giảng viên hướng dẫn — đã dành thời gian định hướng đề tài, góp ý về phạm vi và nhắc nhở những điểm dễ bỏ sót khi làm một hệ thống hoàn chỉnh — đặc biệt là những gì liên quan đến bảo mật và tính đúng đắn của dữ liệu.

Cuối cùng, xin cảm ơn các bạn học viên cùng khóa đã trao đổi, thử nghiệm và góp ý cho sản phẩm trong quá trình thực hiện.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo chắc chắn còn thiếu sót. Em rất mong nhận được nhận xét và góp ý của quý thầy cô để hoàn thiện hơn.

Thành phố Hồ Chí Minh, tháng 8 năm 2026

Học viên thực hiện

VŨ TÂM THIỆN TÍN

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm đề nghị:/10

Thành phố Hồ Chí Minh, ngày tháng năm 2026

Giảng viên hướng dẫn

(ký và ghi rõ họ tên)

MỤC LỤC

DANH MỤC HÌNH VẼ

DANH MỤC BẢNG BIỂU

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ viết tắt	Dạng đầy đủ	Giải thích ngắn
API	Application Programming Interface	Giao diện lập trình ứng dụng — tập hợp các điểm cuối để phần mềm gọi lẫn nhau
BE	Back-end	Phần chạy trên máy chủ: xử lý nghiệp vụ và truy cập cơ sở dữ liệu
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp và triển khai liên tục — tự động kiểm tra rồi đưa mã nguồn lên môi trường thật
CORS	Cross-Origin Resource Sharing	Cơ chế của trình duyệt cho phép hoặc chặn truy cập tài nguyên từ tên miền khác
CRUD	Create, Read, Update, Delete	Bốn thao tác cơ bản trên dữ liệu
CSDL	Cơ sở dữ liệu	Nơi lưu trữ dữ liệu có cấu trúc
ERD	Entity Relationship Diagram	Sơ đồ quan hệ thực thể
FE	Front-end	Phần chạy trong trình duyệt của người dùng
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu dạng văn bản
JWT	JSON Web Token	Chuỗi ký tự có chữ ký số, dùng để xác thực không cần lưu phiên ở máy chủ
LLM	Large Language Model	Mô hình ngôn ngữ lớn — nền tảng của các trợ lý AI sinh nội dung
ORM	Object Relational Mapping	Lớp ánh xạ giữa bảng cơ sở dữ liệu và đối tượng trong mã nguồn
PaaS	Platform as a Service	Nền tảng cho thuê — nhà cung cấp lo hạ tầng, lập trình viên chỉ đưa mã nguồn lên
REST	Representational State Transfer	Kiểu kiến trúc thiết kế API dựa trên tài nguyên và phương thức HTTP
SPA	Single Page Application	Ứng dụng một trang — điều hướng ngay trong trình duyệt, không tải lại toàn trang
SQL	Structured Query Language	Ngôn ngữ truy vấn cơ sở dữ liệu quan hệ
UC	Use Case	Trường hợp sử dụng — một chức năng nhìn từ phía người dùng
XSS	Cross-Site Scripting	Tấn công chèn mã kịch bản độc hại vào trang web

CHƯƠNG 1. TỔNG QUAN

1.1. Lý do chọn đề tài

Học trực tuyến đã trở thành một phần bình thường của giáo dục và đào tạo nội bộ. Tuy nhiên, phần lớn nền tảng thương mại đều được thiết kế cho quy mô lớn, đi kèm thanh toán, tiếp thị và nhiều tính năng không cần thiết đối với một trung tâm đào tạo, một bộ môn hay một nhóm học tập nhỏ. Điều các đơn vị này thực sự cần chỉ gồm ba việc: người dạy soạn được nội dung và bài kiểm tra, người học học theo lộ trình và tự đánh giá được kết quả, người quản lý kiểm soát được chất lượng nội dung trước khi công bố.

Đề tài số 4 của chương trình — *Nền tảng học tập và quiz trực tuyến* — đặt ra đúng ba nhu cầu đó dưới dạng ba vai trò: học viên, giảng viên và quản trị viên. Đề tài vừa đủ nhỏ để hoàn thành trong khuôn khổ một đồ án tốt nghiệp, lại vừa đủ phức tạp để buộc người thực hiện phải giải quyết những vấn đề rất thật của một hệ thống web nhiều người dùng:

- **Phân quyền hai chiều.** Không chỉ phân quyền theo vai trò, mà còn phải phân quyền theo quyền sở hữu tài nguyên: một giảng viên không được sửa khóa học của giảng viên khác.
- **Tính toàn vẹn dữ liệu.** Thứ tự bài học, số lượt làm bài, việc ghi danh trùng — tất cả đều phải được bảo đảm ở tầng cơ sở dữ liệu chứ không chỉ bằng câu lệnh điều kiện trong mã nguồn.
- **Bảo mật nghiệp vụ.** Đáp án đúng của quiz tuyệt đối không được rời khỏi máy chủ trước khi học viên nộp bài; việc chấm điểm phải do máy chủ thực hiện.
- **Quy trình phê duyệt.** Vòng đời một khóa học là một máy trạng thái, không phải một cờ đúng/sai.

Ngoài ra, đề tài yêu cầu tích hợp trí tuệ nhân tạo. Đây là cơ hội để thực hành một nguyên tắc quan trọng trong kỹ thuật phần mềm hiện đại: xem đầu ra của mô hình ngôn ngữ lớn là **một nguồn dữ liệu đầu vào không đáng tin**, phải đi qua đúng bộ kiểm chứng như dữ liệu do con người nhập, và không được phép tự ý ghi vào cơ sở dữ liệu.

1.2. Mục tiêu của đề án

1.2.1. Mục tiêu tổng quát

Xây dựng hoàn chỉnh một nền tảng học tập và kiểm tra trực tuyến mang tên **LearnQuiz**, chạy được trên môi trường Internet thật, phục vụ đủ ba vai trò của đề tài, có tích hợp trợ lý AI, được kiểm thử tự động và triển khai theo quy trình công nghiệp.

1.2.2. Mục tiêu cụ thể

- Thiết kế cơ sở dữ liệu quan hệ chuẩn hóa với các ràng buộc toàn vẹn được đặt ở đúng tầng cơ sở dữ liệu.
- Xây dựng RESTful API có phiên bản, khuôn dạng phản hồi thống nhất, xác thực bằng JWT hai token và phân quyền theo cả vai trò lẫn quyền sở hữu.
- Xây dựng giao diện người dùng dạng ứng dụng một trang, đáp ứng nhiều kích thước màn hình, tự động làm mới phiên đăng nhập khi hết hạn.
- Bảo đảm ba quy tắc nghiệp vụ then chốt: chấm điểm phía máy chủ, khóa nội dung theo lộ trình, và phê duyệt nội dung trước khi công bố.
- Tích hợp Google Gemini cho ba tính năng: sinh câu hỏi trắc nghiệm, giải thích đáp án sai và tóm tắt bài học.
- Viết bộ kiểm thử tự động bao phủ các quy tắc nghiệp vụ quan trọng, chạy được trong môi trường tích hợp liên tục mà không cần dựng cơ sở dữ liệu.
- Đóng gói bằng Docker và triển khai thực tế lên nền tảng đám mây.

1.3. Đối tượng và phạm vi

1.3.1. Đối tượng người dùng

Bảng 1.1. Ba nhóm người dùng của hệ thống

Vai trò	Mô tả	Trọng số điểm theo đề tài
Học viên (student)	Tìm khóa học, ghi danh, học theo lộ trình, làm quiz, theo dõi tiến độ	4,0 / 10
Giảng viên (instructor)	Soạn khóa học, bài học, quiz; gửi khóa học chờ duyệt; xem thống kê lớp	4,0 / 10
Quản trị viên (admin)	Duyệt nội dung, quản lý danh mục và tài khoản, xem thống kê toàn hệ thống	2,0 / 10

1.3.2. Phạm vi chức năng

Ba bảng dưới đây ánh xạ từng yêu cầu trong đề bài sang hạng mục kỹ thuật cụ thể đã được cài đặt. Đây cũng là căn cứ để đối chiếu khi nghiệm thu.

Bảng 1.2. Ánh xạ yêu cầu của vai trò Học viên (4,0 điểm)

#	Yêu cầu đề tài	Hạng mục kỹ thuật đã cài đặt
1	Xem danh sách khóa học, lọc theo danh mục và độ khó	GET /courses với tham số category, level, search, sort, phân trang
2	Ghi danh khóa học miễn phí	POST /courses/:id/enroll, ràng buộc @@unique([studentId, courseId])
3	Học từng bài theo thứ tự (video hoặc văn bản)	Cột order duy nhất trong khóa; hàm computeUnlock khóa bài chưa tới lượt
4	Đánh dấu bài học hoàn thành	PATCH /lessons/:id/complete ghi vào bảng lesson_progress
5	Làm quiz: chọn đáp án, nộp bài, xem điểm ngay	POST /quiz/:id/submit — máy chủ chấm điểm, không tin dữ liệu từ trình duyệt
6	Xem lại đáp án đúng/sai sau khi nộp	GET /submissions/:id chỉ trả isCorrect sau khi bài đã được nộp
7	Xem tiến độ khóa học và điểm trung bình	GET /enrollments/me tổng hợp bằng aggregate của Prisma
8	Làm lại quiz nếu chưa đạt	Bộ ba attemptNo — maxAttempts — passScore
9	AI giải thích đáp án sai	POST /ai/explain-answer, lưu kết quả vào cột answers.ai_explanation

Bảng 1.3. Ánh xạ yêu cầu của vai trò Giảng viên (4,0 điểm)

#	Yêu cầu đề tài	Hạng mục kỹ thuật đã cài đặt
1	Tạo khóa học (tên, mô tả, danh mục, ảnh bìa)	POST /courses, trạng thái luôn bị ép về draft
2	Thêm bài học — thứ tự quan trọng	POST /courses/:id/lessons, ràng buộc @@unique([courseId, order])
3	Tạo quiz: câu hỏi, bốn đáp án, đánh dấu đáp án đúng	PUT /lessons/:id/quiz ghi lồng Question và Choice trong một giao dịch
4	Thống kê học viên: số ghi danh, điểm trung bình lớp	GET /courses/:id/stats dùng groupBy kết hợp _avg
5	Sửa, xóa bài học và câu hỏi	PATCH / DELETE kèm kiểm tra quyền sở hữu tài nguyên

#	Yêu cầu đề tài	Hạng mục kỹ thuật đã cài đặt
6	AI hỗ trợ soạn quiz	POST /ai/lessons/:id/generate-quiz trả bản nháp, không tự ghi vào CSDL

Bảng 1.4. Ảnh xạ yêu cầu của vai trò Quản trị viên (2,0 điểm)

#	Yêu cầu đề tài	Hạng mục kỹ thuật đã cài đặt
1	Duyệt khóa học trước khi hiển thị công khai	PATCH /courses/:id/publish và /reject — máy trạng thái bốn bước
2	Quản lý danh mục (CRUD)	Nhóm /categories với đủ năm điểm cuối
3	Quản lý người dùng, khóa tài khoản vi phạm	PATCH /users/:id/status — cờ is_active chặn ngay tại bước đăng nhập
4	Thống kê tổng quan hệ thống	GET /admin/stats — người dùng theo vai trò, khóa học theo trạng thái, top 5 khóa đông học viên nhất

1.3.3. Giới hạn phạm vi

Những hạng mục sau **có chủ đích không thực hiện**, vì không nằm trong yêu cầu của đề tài và sẽ làm loãng trọng tâm:

- **Thanh toán và khóa học trả phí** — đề tài quy định rõ “*ghi danh khóa học miễn phí*”.
- **Tải lên và truyền phát video** — bài học lưu địa chỉ video bên ngoài (YouTube, Vimeo), hệ thống không tự lưu trữ tệp video.
- **Diễn đàn, trò chuyện, thông báo qua thư điện tử** — thuộc phạm vi các đề tài khác của chương trình.
- **Ứng dụng di động gốc** — phạm vi là ứng dụng web đáp ứng nhiều kích thước màn hình.
- **Quiz dạng tự luận** — đề tài quy định trắc nghiệm bốn đáp án, một đáp án đúng.

1.4. Phương pháp thực hiện

Đồ án được thực hiện theo quy trình Scrum rút gọn, chia thành sáu chặng phát triển (sprint). Mỗi sprint có một mục tiêu nghiệp vụ trọn vẹn và kết thúc bằng một sản phẩm chạy được, thay vì chia theo tầng kỹ thuật (làm hết cơ sở dữ liệu rồi mới làm giao diện). Cách chia này giúp phát hiện sai sót thiết kế từ sớm, khi chi phí sửa còn thấp.

Bảng 1.5. Kế hoạch sáu sprint và kết quả bàn giao

Sprint	Mục tiêu	Kết quả bàn giao
0	Nền móng	Khung dự án dạng monorepo; lược đồ 11 bảng; đăng ký, đăng nhập, làm mới token, phân quyền; dữ liệu mẫu
1	Danh mục và khóa học	CRUD danh mục; CRUD khóa học và gửi duyệt; trang danh sách có lọc, sắp xếp, phân trang; trang chi tiết
2	Bài học và tiến độ	Soạn và sắp xếp bài học; ghi danh; màn hình học bài; khóa bài theo lộ trình; thanh tiến độ
3	Quiz	Trình soạn quiz động; làm bài; chấm điểm phía máy chủ; xem lại đáp án; giới hạn số lượt; 111 phép kiểm thử
4	Quản trị và thống kê	Máy trạng thái duyệt khóa học; quản lý người dùng; trang thống kê lớp và toàn hệ thống; 253 phép kiểm thử
5	AI và triển khai	Ba tính năng Gemini; Docker hai giai đoạn; GitHub Actions; cấu hình Render và Vercel; 344 phép kiểm thử

Định nghĩa **“hoàn thành”** áp dụng cho mọi sprint: lệnh biên dịch sạch lỗi ở cả hai dự án, mọi điểm cuối mới đã thử bằng công cụ gọi API, không còn lệnh in ra màn hình sót lại trong mã nguồn, và toàn bộ bộ kiểm thử tự động vẫn xanh.

1.5. Kết quả đạt được

Toàn bộ sáu sprint đã hoàn tất. Bảng dưới đây tóm tắt quy mô sản phẩm bàn giao.

Bảng 1.6. Quy mô sản phẩm bàn giao

Hạng mục	Số liệu
Tổng số tệp mã nguồn và cấu hình (không kể thư viện ngoài)	143 tệp
Mã nguồn back-end (TypeScript)	4.017 dòng / 60 tệp
Mã nguồn front-end (TypeScript và TSX)	6.067 dòng / 56 tệp
Mã nguồn kiểm thử tự động	1.609 dòng / 10 tệp
Lược đồ cơ sở dữ liệu	11 bảng, 3 kiểu liệt kê, 217 dòng
Điểm cuối API (tính cả biến thể phương thức)	46 điểm cuối trên <code>/api/v1</code> , cộng <code>/health</code>
Phép khẳng định trong bộ kiểm thử tự động	344 phép — đạt 344/344
Màn hình giao diện	21 màn hình (công khai, học viên, giảng viên, quản trị)

1.6. Bố cục báo cáo

Chương 1 trình bày lý do chọn đề tài, mục tiêu, phạm vi, phương pháp thực hiện và tóm tắt kết quả.

Chương 2 hệ thống hóa cơ sở lý thuyết và công nghệ được sử dụng, kèm lý do lựa chọn từng thành phần.

Chương 3 phân tích yêu cầu và trình bày thiết kế: mô hình use-case, kiến trúc, cơ sở dữ liệu, giao diện lập trình và vòng đời khóa học.

Chương 4 mô tả cách cài đặt hệ thống, tập trung vào những đoạn mã và quyết định kỹ thuật có ý nghĩa quyết định đối với tính đúng đắn và an toàn.

Chương 5 trình bày chiến lược kiểm thử, kết quả kiểm thử, quy trình đóng gói, tích hợp liên tục và triển khai.

Chương 6 kết luận, nêu hạn chế và hướng phát triển. Cuối báo cáo là tài liệu tham khảo và bốn phụ lục.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

Chương này trình bày những khái niệm nền tảng mà thiết kế ở Chương 3 và phần cài đặt ở Chương 4 dựa vào. Mỗi mục đều nêu rõ vì sao lựa chọn đó phù hợp với đề tài, thay vì chỉ liệt kê đặc điểm công nghệ.

2.1. Kiến trúc client–server tách rời

Trong kiến trúc web truyền thống, máy chủ dựng sẵn trang HTML rồi gửi về trình duyệt. Kiến trúc tách rời (*decoupled*) chia hệ thống thành hai chương trình độc lập: một ứng dụng chạy trong trình duyệt chịu trách nhiệm hiển thị, và một máy chủ chỉ trả dữ liệu thuần dưới dạng JSON. Hai bên giao tiếp qua HTTP.

Cách chia này mang lại ba lợi ích trực tiếp cho đề tài. Thứ nhất, hai phần có thể được phát triển, kiểm thử và triển khai độc lập — trong đồ án này front-end nằm trên Vercel còn back-end nằm trên Render. Thứ hai, cùng một API có thể phục vụ thêm ứng dụng di động về sau mà không phải viết lại nghiệp vụ. Thứ ba — và quan trọng nhất về mặt an toàn — ranh giới giữa hai bên trở nên rõ ràng: mọi thứ chạy trong trình duyệt đều nằm trong tầm kiểm soát của người dùng, nên **mọi quyết định về quyền và điểm số bắt buộc phải nằm ở phía máy chủ**.

Ứng dụng một trang (SPA) là hệ quả tự nhiên: trình duyệt tải một lần bộ mã JavaScript, sau đó tự dựng lại giao diện khi người dùng điều hướng và chỉ gọi API để lấy dữ liệu. Đối lại, ứng dụng phải tự quản lý định tuyến, trạng thái đăng nhập và xử lý lỗi mạng — những phần được trình bày ở mục 4.9.

2.2. RESTful API

REST là kiểu kiến trúc thiết kế giao diện lập trình dựa trên bốn nguyên tắc mà đồ án tuân thủ chặt chẽ:

- **Định danh theo tài nguyên.** Đường dẫn mô tả *danh từ* chứ không mô tả *hành động*: `/courses/5/lessons` thay vì `/getLessonsOfCourse?id=5`.
- **Phương thức HTTP mang ngữ nghĩa.** GET đọc, POST tạo mới, PUT thay thế trọn gói, PATCH sửa một phần, DELETE xóa.

- **Phi trạng thái.** Máy chủ không lưu phiên làm việc; mỗi yêu cầu tự mang theo thông tin xác thực. Nhờ vậy có thể nhân bản nhiều tiến trình máy chủ mà không cần chia sẻ bộ nhớ.
- **Mã trạng thái đúng ngữ nghĩa.** Máy khách phân biệt được lỗi do mình gửi sai với lỗi do máy chủ hỏng.

Bảng 2.1. Các mã trạng thái HTTP được hệ thống sử dụng

Mã	Ý nghĩa	Trường hợp phát sinh trong LearnQuiz
200	OK	Đọc dữ liệu hoặc cập nhật thành công
201	Created	Tạo khóa học, bài học, ghi danh, nộp bài quiz
400	Bad Request	Dữ liệu gửi lên không qua được kiểm chứng Yup
401	Unauthorized	Thiếu token, token sai hoặc đã hết hạn
403	Forbidden	Đã đăng nhập nhưng không đủ quyền, hoặc không phải chủ sở hữu tài nguyên
404	Not Found	Tài nguyên không tồn tại; Prisma ném P2025
409	Conflict	Ghi danh trùng, trùng thứ tự bài học, sửa đề khi đã có bài nộp; Prisma ném P2002
422	Unprocessable Entity	AI trả về dữ liệu không qua được kiểm chứng
429	Too Many Requests	Vượt giới hạn tần suất gọi
503	Service Unavailable	Chưa cấu hình khóa API của Gemini, hoặc dịch vụ ngoài đang lỗi

Hệ thống đặt toàn bộ API dưới tiền tố `/api/v1`. Việc gắn số phiên bản ngay từ đầu cho phép về sau sửa đổi khuôn dạng dữ liệu ở `/api/v2` mà không làm hỏng các máy khách cũ.

2.3. Xác thực và phân quyền bằng JWT

JSON Web Token là một chuỗi gồm ba phần ngăn cách bởi dấu chấm: phần đầu mô tả thuật toán, phần thân chứa dữ liệu (ở đây là mã người dùng và vai trò), phần cuối là chữ ký số tạo bằng khóa bí mật của máy chủ. Máy chủ không cần tra cơ sở dữ liệu vẫn xác minh được token là thật, vì chỉ máy chủ mới ký được chữ ký đó.

Điểm yếu cố hữu của JWT là **không thu hồi được**: token đã phát ra sẽ còn hiệu lực cho tới khi hết hạn. Hệ thống khắc phục bằng mô hình hai token:

- **Access token** sống ngắn (15 phút), gửi kèm mọi yêu cầu. Nếu bị lộ, thiệt hại giới hạn trong 15 phút.
- **Refresh token** sống dài (7 ngày), **được lưu trong cơ sở dữ liệu**. Vì có bản lưu ở máy chủ nên có thể xóa đi để vô hiệu hóa ngay lập tức — đây chính là cơ chế đăng xuất và cơ chế khóa tài khoản.

Phân quyền được tách thành hai tầng độc lập. Tầng thứ nhất kiểm tra **vai trò** (role): chỉ quản trị viên mới được duyệt khóa học. Tầng thứ hai kiểm tra **quyền sở hữu tài nguyên**: một giảng viên chỉ sửa được khóa học do chính mình tạo. Thiếu tầng thứ hai, mọi giảng viên trong hệ thống đều sửa được nội dung của nhau — một lỗi hổng thường gặp trong các hệ thống chỉ phân quyền theo vai trò.

2.4. Node.js và Express — mô hình middleware

Node.js cho phép chạy JavaScript ngoài trình duyệt theo mô hình một luồng, không chặn, hướng sự kiện. Với ứng dụng web thiên về chờ đợi vào-ra (truy vấn cơ sở dữ liệu, gọi dịch vụ ngoài) thay vì tính toán nặng, mô hình này cho thông lượng cao trên tài nguyên khiêm tốn — phù hợp với gói miễn phí dùng để bảo vệ đồ án.

Express tổ chức việc xử lý một yêu cầu thành **chuỗi middleware**: mỗi mắt xích nhận (req, res, next), làm một việc nhỏ, rồi hoặc trả lời, hoặc gọi next() để chuyển tiếp, hoặc gọi next(err) để nhảy thẳng tới bộ xử lý lỗi. Ưu điểm sự phạm và kỹ thuật của mô hình này là **thứ tự các mối quan tâm hiện ra rõ ràng trong mã nguồn**: bảo mật header, kiểm tra nguồn gọi, giới hạn tần suất, ghi nhật ký, xác thực, phân quyền, kiểm chứng dữ liệu — rồi mới tới nghiệp vụ. Chi tiết chuỗi này được trình bày ở mục 4.2.

2.5. TypeScript

TypeScript bổ sung hệ thống kiểu tĩnh cho JavaScript và biên dịch ra JavaScript thuần. Đồ án dùng TypeScript ở **cả hai đầu** với ba lý do.

Thứ nhất, phần lớn lỗi khi làm việc với dữ liệu quan hệ là lỗi truy cập sai tên trường hoặc sai kiểu — những lỗi này bị bắt ngay lúc biên dịch thay vì lúc chạy. Thứ hai, Prisma sinh ra kiểu dữ liệu trực tiếp từ lược đồ cơ sở dữ liệu, nên sửa lược đồ mà quên sửa mã nguồn sẽ làm dự án không biên dịch được — một cơ chế cảnh báo miễn phí. Thứ

ba, kiểu dữ liệu mô tả phản hồi API được khai báo một lần và dùng chung ở cả hai phía, giúp hai đầu không hiểu khác nhau về hình dạng dữ liệu.

2.6. Prisma ORM và PostgreSQL

PostgreSQL được chọn vì đề tài xoay quanh quan hệ giữa các thực thể và các ràng buộc duy nhất phức hợp — đúng thể mạnh của cơ sở dữ liệu quan hệ. Một cơ sở dữ liệu dạng tài liệu sẽ phải mô phỏng lại bằng mã nguồn những gì PostgreSQL làm sẵn ở tầng lưu trữ.

Prisma đóng vai trò lớp ánh xạ đối tượng – quan hệ với ba đặc điểm được khai thác triệt để trong đồ án:

- **Lược đồ là nguồn sự thật duy nhất.** Tập `schema.prisma` mô tả toàn bộ cấu trúc; từ đó Prisma sinh ra cả migration lẫn mã nguồn có kiểu.
- **Migration có phiên bản.** Mỗi thay đổi cấu trúc là một tệp SQL được lưu trong kho mã nguồn, chạy lại được trên môi trường thật bằng `prisma migrate deploy`.
- **Truy vấn tham số hóa.** Dữ liệu người dùng không bao giờ được nối thẳng vào câu lệnh SQL, nên nguy cơ tiêm nhiễm SQL bị loại bỏ ngay từ thiết kế.

Ngoài ra, cơ chế `select` và `include` của Prisma cho phép chỉ định chính xác những cột được lấy lên. Đây không chỉ là tối ưu hiệu năng: trong đồ án này, nó chính là **biện pháp kỹ thuật bảo đảm cờ đáp án đúng không rời khỏi máy chủ** (mục 4.4.1).

2.7. React và hệ sinh thái phía trình duyệt

React xây dựng giao diện từ các thành phần (component) độc lập, mỗi thành phần tự quản lý trạng thái riêng và được vẽ lại khi trạng thái thay đổi. Hệ thống dùng React 19 cùng bốn thư viện hỗ trợ:

Bảng 2.2. Thư viện phía trình duyệt và vai trò

Thư viện	Vai trò trong hệ thống
Vite	Công cụ dựng dự án: khởi động và nạp lại tức thì khi phát triển, gói tối ưu khi phát hành
MUI (Material UI)	Bộ thành phần giao diện dựng sẵn, đáp ứng nhiều kích thước màn hình

Thư viện	Vai trò trong hệ thống
React Router	Định tuyến phía trình duyệt; kết hợp <code>ProtectedRoute</code> để chặn truy cập theo vai trò
React Hook Form + Yup	Quản lý biểu mẫu ít vẽ lại; dùng chung quy tắc kiểm chứng với máy chủ
axios	Gọi HTTP; bộ chặn (interceptor) tự gắn token và tự làm mới khi hết hạn

2.8. Kiểm chứng dữ liệu bằng Yup

Yup cho phép mô tả quy tắc dữ liệu dưới dạng khai báo, ví dụ: câu hỏi phải có đúng bốn đáp án và đúng một đáp án được đánh dấu đúng. Đồ án dùng **cùng một thư viện ở cả hai đầu**: phía trình duyệt để báo lỗi ngay khi người dùng gõ, phía máy chủ để bảo vệ dữ liệu thật.

Cần nhấn mạnh: kiểm chứng ở trình duyệt **không phải là bảo mật** — người dùng có thể gọi thẳng API bằng công cụ dòng lệnh. Kiểm chứng phía máy chủ mới là lớp bảo vệ thật; kiểm chứng phía trình duyệt chỉ để trải nghiệm tốt hơn. Ngoài ra, tùy chọn `stripUnknown` của Yup loại bỏ mọi trường lạ trong dữ liệu gửi lên, qua đó chặn kiểu tấn công gán tràn thuộc tính (*mass assignment*) — chẳng hạn gửi kèm `role: "admin"` khi đăng ký tài khoản.

2.9. Docker, tích hợp liên tục và mô hình triển khai

Docker đóng gói ứng dụng cùng toàn bộ môi trường chạy vào một *image*, loại bỏ tình trạng “máy em chạy được mà máy thầy thì không”. Đồ án dùng kỹ thuật **dựng nhiều giai đoạn**: giai đoạn đầu biên dịch mã nguồn, giai đoạn sau chỉ chép kết quả biên dịch sang một ảnh nền sạch. Ảnh cuối cùng do đó không chứa mã TypeScript, không chứa thư viện chỉ dùng khi phát triển, và tiến trình chạy dưới người dùng thường thay vì `root`.

Tích hợp liên tục (CI) là cơ chế tự động chạy kiểm tra mỗi khi mã nguồn được đẩy lên kho chung. Nguyên tắc áp dụng trong đồ án: **không để mã hỏng lọt vào nhánh chính**. Ba nhóm kiểm tra được chạy song song — back-end, front-end và rà soát bảo mật (mục 5.6).

Về triển khai, đồ án dùng mô hình nền tảng cho thuê (PaaS): Render chạy máy chủ API cùng cơ sở dữ liệu PostgreSQL, Vercel phục vụ tập tĩnh của front-end qua mạng phân phối nội dung. Cả hai đều có gói miễn phí đủ cho mục đích trình bày đồ án.

2.10. Mô hình ngôn ngữ lớn và Google Gemini API

Mô hình ngôn ngữ lớn (LLM) sinh văn bản theo xác suất dựa trên ngữ cảnh được cung cấp. Hai đặc tính của LLM chi phối toàn bộ thiết kế phần AI trong đồ án:

- **Đầu ra không xác định.** Cùng một câu lệnh có thể cho kết quả khác nhau, và mô hình có thể “bịa” — sinh ra nội dung nghe hợp lý nhưng sai.
- **Chi phí và hạn mức theo lượt gọi.** Mỗi lời gọi đều tốn tiền và có giới hạn tần suất.

Từ đó rút ra bốn nguyên tắc thiết kế, được cài đặt ở mục 4.8: (1) buộc mô hình trả về JSON theo lược đồ định sẵn thông qua tham số `responseMimeType` và `responseSchema`; (2) kiểm chứng đầu ra bằng **đúng bộ quy tắc dùng cho dữ liệu người nhập tay**; (3) không cho AI ghi thẳng vào cơ sở dữ liệu — kết quả chỉ là bản nháp để con người duyệt; (4) lưu lại kết quả đã sinh để lần sau không phải gọi lại.

Khóa API của Gemini **chỉ tồn tại trong biến môi trường của máy chủ**. Đây là ràng buộc bắt buộc: mọi biến môi trường có tiền tố `VITE_` đều được nhúng thẳng vào gói JavaScript tải về trình duyệt, nên đặt khóa ở front-end đồng nghĩa với công bố khóa cho toàn thế giới.

2.11. Tổng hợp lựa chọn công nghệ

Bảng 2.3. Tổng hợp công nghệ sử dụng và lý do lựa chọn

Lớp	Công nghệ (phiên bản)	Lý do lựa chọn
Ngôn ngữ	TypeScript 5.6 / 5.9	Bắt lỗi kiểu lúc biên dịch; dùng chung kiểu dữ liệu API giữa hai đầu
Máy chủ	Node.js 22, Express 4.21	Đúng yêu cầu đề tài; chuỗi middleware rõ ràng, dễ trình bày và kiểm thử
ORM	Prisma 5.22	Lược đồ là nguồn sự thật; migration có phiên bản; <code>select</code> chặn rò rỉ dữ liệu
Cơ sở dữ liệu	PostgreSQL 16 (Docker cục bộ) / 18 (Render production)	Quan hệ và ràng buộc duy nhất phức hợp là trung tâm của đề tài; Prisma tương thích cả hai phiên bản

Lớp	Công nghệ (phiên bản)	Lý do lựa chọn
Kiểm chứng	Yup 1.4	Một thư viện dùng cho cả hai đầu; stripUnknown chặn gán tràn thuộc tính
Xác thực	jsonwebtoken 9, bcryptjs 2.4	Phi trạng thái, phù hợp kiến trúc tách rời; băm mật khẩu có muối
Giao diện	React 19, Vite 8, MUI 9	Đúng yêu cầu đề tài; dựng nhanh, có sẵn thành phần đáp ứng đa màn hình
Biểu mẫu	React Hook Form 7 + Yup	Ít vẽ lại; quy tắc kiểm chứng đồng bộ với máy chủ
Gọi HTTP	axios 1.18	Bộ chặn cho phép tự gán và tự làm mới token
Nhật ký	pino 9	Nhật ký JSON có cấu trúc, tự che các trường nhạy cảm
Bảo mật	helmet 8, express-rate-limit 7	Đặt header phòng vệ; giới hạn tần suất chống dò mật khẩu
Đóng gói	Docker (dựng hai giai đoạn)	Môi trường chạy đồng nhất; ảnh gọn, không chạy bằng root
Triển khai	Render, Vercel, GitHub Actions	Đúng yêu cầu đề tài; gói miễn phí đủ cho demo; CI chặn mã hỏng
Trí tuệ nhân tạo	Google Gemini 3.6 Flash	Đúng yêu cầu đề tài; hỗ trợ trả JSON theo lược đồ; chỉ gọi từ máy chủ (model 2.0 Flash ban đầu bị Google ngừng hỗ trợ từ 01/06/2026, đã nâng cấp)

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Xác định yêu cầu

3.1.1. Yêu cầu chức năng

Từ đề bài, hệ thống được phân rã thành 28 trường hợp sử dụng, nhóm theo bảy nhóm chức năng. Mã use-case được dùng xuyên suốt báo cáo để đối chiếu giữa thiết kế, cài đặt và kiểm thử.

Bảng 3.1. Danh sách trường hợp sử dụng

Mã	Tên trường hợp sử dụng	Tác nhân	Mức ưu tiên
UC-01	Đăng ký tài khoản	Khách	Cao
UC-02	Đăng nhập	Cả ba vai trò	Cao
UC-03	Làm mới phiên đăng nhập tự động	Cả ba vai trò	Cao
UC-04	Đăng xuất và thu hồi phiên	Cả ba vai trò	Trung bình
UC-05	Xem danh sách khóa học công khai	Khách, Học viên	Cao
UC-06	Lọc, tìm kiếm và sắp xếp khóa học	Khách, Học viên	Cao
UC-07	Ghi danh khóa học miễn phí	Học viên	Cao
UC-08	Xem lộ trình và nội dung bài học	Học viên	Cao
UC-09	Đánh dấu bài học hoàn thành	Học viên	Cao
UC-10	Theo dõi tiến độ khóa học	Học viên	Cao
UC-11	Làm bài quiz	Học viên	Cao
UC-12	Nộp bài và nhận điểm	Học viên	Cao
UC-13	Xem lại đáp án đúng và sai	Học viên	Cao
UC-14	Làm lại quiz trong giới hạn số lượt	Học viên	Trung bình
UC-15	Tạo và sửa khóa học	Giảng viên	Cao
UC-16	Soạn bài học và sắp xếp thứ tự	Giảng viên	Cao
UC-17	Soạn quiz cho bài học	Giảng viên	Cao
UC-18	Gửi khóa học chờ duyệt	Giảng viên	Cao
UC-19	Xem thống kê lớp học	Giảng viên	Trung bình
UC-20	Xóa bài học, câu hỏi, khóa học	Giảng viên	Trung bình
UC-21	Duyệt hoặc từ chối khóa học	Quản trị viên	Cao

Mã	Tên trường hợp sử dụng	Tác nhân	Mức ưu tiên
UC-22	Gỡ khóa học đang công khai	Quản trị viên	Trung bình
UC-23	Quản lý danh mục	Quản trị viên	Trung bình
UC-24	Quản lý và khóa tài khoản người dùng	Quản trị viên	Cao
UC-25	Xem thống kê toàn hệ thống	Quản trị viên	Trung bình
UC-26	Sinh câu hỏi trắc nghiệm bằng AI	Giảng viên	Cao
UC-27	Nhờ AI giải thích đáp án sai	Học viên	Cao
UC-28	Nhờ AI tóm tắt bài học	Học viên, Giảng viên	Thấp

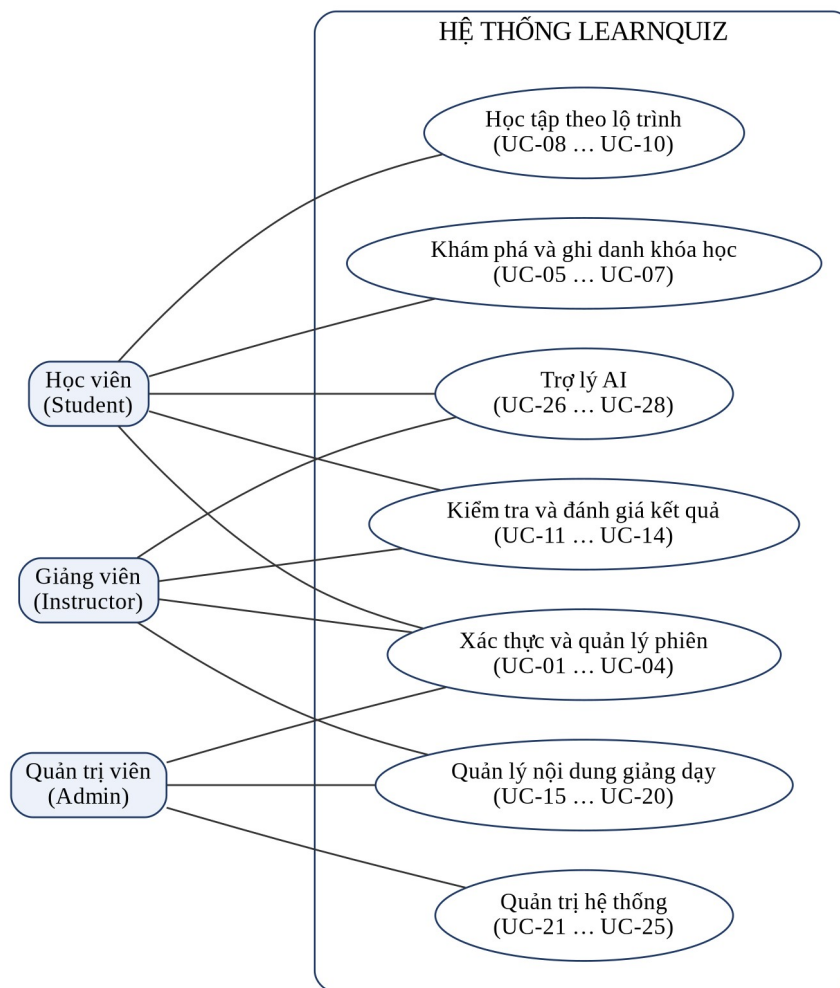
3.1.2. Yêu cầu phi chức năng

Bảng 3.2. Yêu cầu phi chức năng và tiêu chí nghiệm thu

Nhóm	Yêu cầu	Tiêu chí nghiệm thu
Bảo mật	Đáp án đúng không rời khỏi máy chủ trước khi nộp bài	Phản hồi HTTP của GET /lessons/:id/quiz không chứa chuỗi isCorrect
Bảo mật	Mật khẩu và mã làm mới phiên không bao giờ xuất hiện trong phản hồi	Kiểm thử tự động rà toàn bộ phản hồi của GET /users
Bảo mật	Chống dò mật khẩu tự động	Giới hạn 20 lượt gọi trong 15 phút cho nhóm /auth
Toàn vẹn	Không tồn tại ghi danh trùng, thứ tự bài học trùng, lượt nộp trùng	Ràng buộc duy nhất ở tầng cơ sở dữ liệu, không chỉ ở tầng mã nguồn
Hiệu năng	Số truy vấn khi lấy thống kê không tăng theo sĩ số lớp	Dùng groupBy; ba truy vấn tổng hợp bất kể lớp có bao nhiêu học viên
Khả dụng	Hệ thống vẫn hoạt động khi dịch vụ AI hỏng hoặc chưa cấu hình	Các nút AI bị làm mờ kèm giải thích; phần còn lại chạy bình thường
Bảo trì	Nghiệp vụ tách khỏi HTTP để kiểm thử độc lập	Ba mô-đun hàm thuần không nhập khẩu Prisma hay Express
Quan sát	Truy vết được mọi yêu cầu và lỗi	Nhật ký JSON có method, path, status, thời gian xử lý; tự che trường nhạy cảm
Giao diện	Dùng được trên điện thoại và máy tính	Bố cục MUI đáp ứng; kiểm tra ở ba mốc chiều rộng màn hình

Nhóm	Yêu cầu	Tiêu chí nghiệm thu
Triển khai	Đưa lên môi trường thật bằng quy trình lặp lại được	Tập render .yaml, vercel.json và quy trình CI ba nhóm kiểm tra

3.2. Mô hình trường hợp sử dụng



Hình 3.1. Sơ đồ trường hợp sử dụng tổng quát

Ba tác nhân đều đi qua nhóm *Xác thực và quản lý phiên*. Quản trị viên kế thừa quyền của giảng viên trong nhóm *Quản lý nội dung giảng dạy* — điều này cho phép quản trị viên xem trước nội dung khi cần thẩm định, nhưng vẫn **không** được tự duyệt khóa học do chính mình tạo (mục 3.6).

Ba trường hợp sử dụng tiêu biểu được đặc tả chi tiết dưới đây: một trường hợp thuộc nghiệp vụ trọng tâm, một thuộc quy trình quản trị và một thuộc phần tích hợp AI.

Bảng 3.3. Đặc tả UC-12 — Nộp bài và nhận điểm

Mục	Nội dung
Tác nhân	Học viên đã ghi danh khóa học
Mục tiêu	Nộp bài làm và nhận điểm số chính xác do máy chủ chấm
Tiền điều kiện	Đã đăng nhập; đã ghi danh; bài học chứa quiz đã được mở khóa; còn lượt làm bài
Hậu điều kiện	Một bản ghi <code>quiz_submissions</code> và các bản ghi <code>answers</code> tương ứng được lưu trong cùng một giao dịch
Luồng chính	- Học viên chọn đáp án cho từng câu hỏi. - Học viên bấm nộp bài. - Hệ thống kiểm chứng cấu trúc bài làm bằng Yup. - Hệ thống kiểm tra ghi danh và số lượt còn lại. - Hệ thống đọc đề kèm cờ đáp án đúng — chỉ trong bộ nhớ máy chủ. - Hệ thống gọi hàm chấm điểm thuần và tính điểm trên thang 0–100. - Hệ thống lưu lượt nộp và từng câu trả lời trong một giao dịch. - Hệ thống trả về điểm, kết luận đạt hay chưa đạt, và đáp án đúng.
Luồng thay thế	4a. Hết lượt làm bài → trả mã 409 kèm thông báo. 3a. Bài làm rỗng hoặc trùng mã câu hỏi → trả mã 400. 5a. Người gọi không phải học viên đã ghi danh → trả mã 403.
Quy tắc nghiệp vụ	Điểm chỉ do máy chủ tính. Trường <code>score</code> nếu có trong dữ liệu gửi lên sẽ bị loại bỏ. Mã đáp án thuộc câu hỏi khác được coi như bỏ trống. Câu không trả lời vẫn tính vào mẫu số.

Bảng 3.4. Đặc tả UC-21 — Duyệt hoặc từ chối khóa học

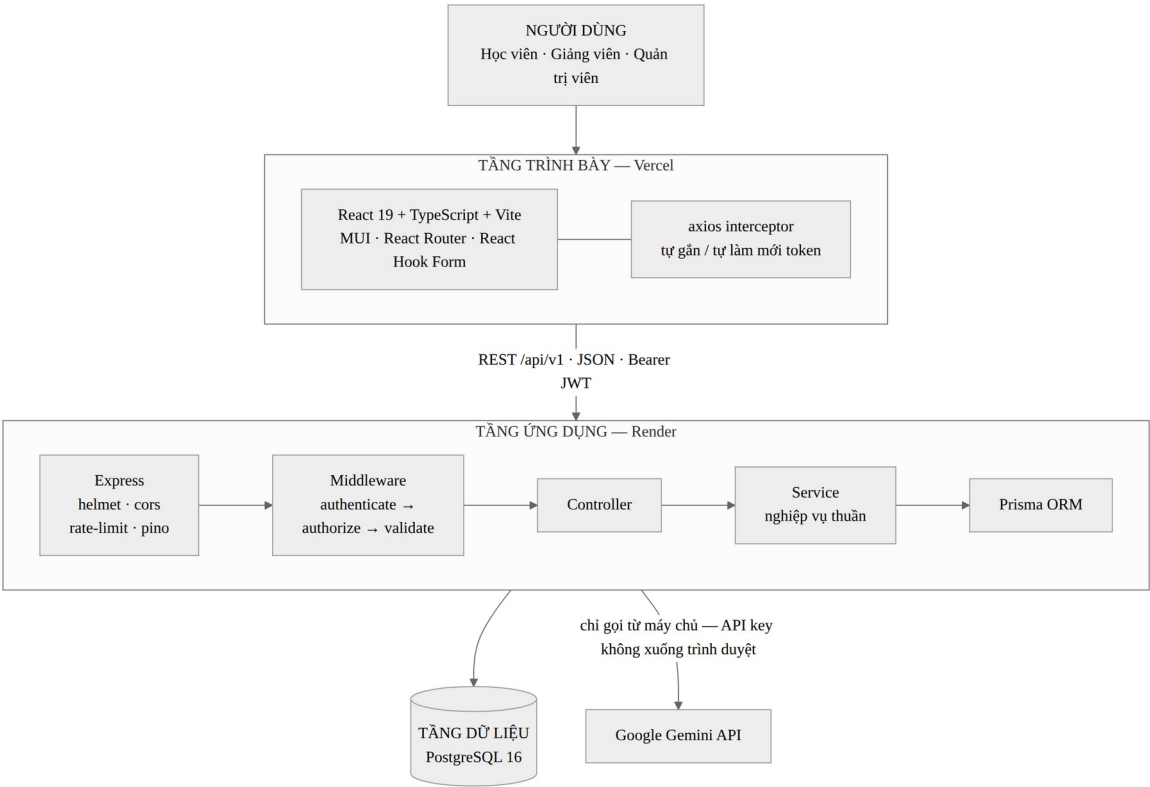
Mục	Nội dung
Tác nhân	Quản trị viên
Mục tiêu	Kiểm soát chất lượng nội dung trước khi khóa học hiển thị công khai
Tiền điều kiện	Đã đăng nhập với vai trò quản trị; khóa học đang ở trạng thái <i>Chờ duyệt</i>
Hậu điều kiện	Khóa học chuyển sang <i>Đang hiển thị công khai</i> kèm mốc thời gian, hoặc <i>Bị từ chối</i> kèm lý do
Luồng chính	- Quản trị viên mở hàng đợi duyệt, khóa chờ lâu nhất xếp đầu. - Quản trị viên xem nội dung khóa học và các bài học. - Quản trị viên chọn duyệt. - Hệ thống đối chiếu bảng chuyển trạng thái, cập nhật trạng thái và ghi mốc công bố.
Luồng thay thế	3a. Chọn từ chối → hệ thống bắt buộc nhập lý do dài ít nhất 10 ký tự, lưu vào cột <code>reject_reason</code>. 4a. Khóa học không ở trạng thái <i>Chờ duyệt</i> → trả mã 409 kèm giải thích trạng thái hiện tại.
Quy tắc nghiệp vụ	Giảng viên — kể cả khi có quyền quản lý khóa học đó — không được tự duyệt khóa học của mình; hệ thống trả mã 403.

Bảng 3.5. Đặc tả UC-26 — Sinh câu hỏi trắc nghiệm bằng AI

Mục	Nội dung
Tác nhân	Giảng viên sở hữu bài học (hoặc quản trị viên)
Mục tiêu	Rút ngắn thời gian soạn đề bằng cách để mô hình ngôn ngữ đề xuất bản nháp câu hỏi
Tiền điều kiện	Bài học đã có nội dung văn bản; máy chủ đã cấu hình khóa API của Gemini
Hậu điều kiện	Không thay đổi cơ sở dữ liệu. Bản nháp chỉ được trả về giao diện để giảng viên duyệt
Luồng chính	- Giảng viên chọn số câu muốn sinh (từ 1 đến 10). - Hệ thống kiểm tra quyền sở hữu bài học và giới hạn tần suất gọi AI. - Hệ thống gửi nội dung bài học kèm lược đồ JSON mong muốn tới Gemini. - Hệ thống kiểm chứng kết quả bằng đúng bộ quy tắc dùng cho câu hỏi nhập tay. - Hệ thống trả bản nháp; giảng viên sửa rồi mới lưu qua chức năng soạn quiz thông thường.
Luồng thay thế	2a. Chưa cấu hình khóa API → trả mã 503 kèm hướng dẫn, giao diện làm mờ nút AI. 3a. Quá 20 giây không phản hồi → hủy lời gọi, trả mã 504. 3b. Vượt hạn mức của Gemini → trả mã 429. 4a. Kết quả không đủ bốn đáp án hoặc có nhiều hơn một đáp án đúng → trả mã 422, không hiển thị cho giảng viên.

3.3. Thiết kế kiến trúc

3.3.1. Kiến trúc tổng thể



Hình 3.2. Kiến trúc tổng thể ba tầng

Hệ thống được tổ chức thành ba tầng triển khai độc lập. Tầng trình bày là ứng dụng một trang chạy trong trình duyệt, được phục vụ dưới dạng tệp tĩnh qua mạng phân phối nội dung của Vercel. Tầng ứng dụng là máy chủ Express chạy trên Render, chịu toàn bộ trách nhiệm về nghiệp vụ, quyền hạn và tính đúng đắn của dữ liệu. Tầng dữ liệu là PostgreSQL do Render quản lý, chỉ nhận kết nối từ tầng ứng dụng.

Dịch vụ Gemini nằm ngoài hệ thống và **chỉ được gọi từ tầng ứng dụng**. Trình duyệt không bao giờ liên lạc trực tiếp với Gemini, nhờ đó khóa API không rời khỏi máy chủ.

3.3.2. Phân tầng bên trong máy chủ

Bên trong máy chủ, mã nguồn được chia thành các tầng có trách nhiệm tách bạch:

Bảng 3.6. Trách nhiệm của từng tầng trong máy chủ

Tầng	Thư mục	Trách nhiệm	Được phép biết
Định tuyến	routes/	Khai báo điểm cuối và lắp chuỗi middleware	HTTP
Middleware	middleware/	Xác thực, phân quyền, kiểm chứng, giới hạn tần suất, xử lý lỗi	HTTP
Điều khiển	controllers/	Đọc tham số, gọi service, định dạng phản hồi	HTTP
Nghiệp vụ	services/	Toàn bộ quy tắc nghiệp vụ và truy cập dữ liệu	Không biết HTTP
Kiểm chứng	schemas/	Mô tả quy tắc dữ liệu bằng Yup	Không biết HTTP
Hạ tầng	utils/	Kết nối Prisma, ký và giải mã JWT, nhật ký, sinh chuỗi định danh	—

Nguyên tắc bất di bất dịch: **tầng điều khiển không chứa nghiệp vụ, tầng nghiệp vụ không biết gì về HTTP**. Nhờ vậy các hàm nghiệp vụ được kiểm thử trực tiếp mà không cần dựng máy chủ; ba mô-đun quizGrader, lessonRules và courseWorkflow thậm chí không nhập khẩu Prisma nên kiểm thử được mà không cần cơ sở dữ liệu.

3.3.3. Khuôn dạng phản hồi thống nhất

Mọi phản hồi của API — thành công hay thất bại — đều theo một khuôn dạng duy nhất. Nhờ đó phía trình duyệt chỉ cần một hàm xử lý lỗi dùng chung cho toàn bộ ứng dụng.

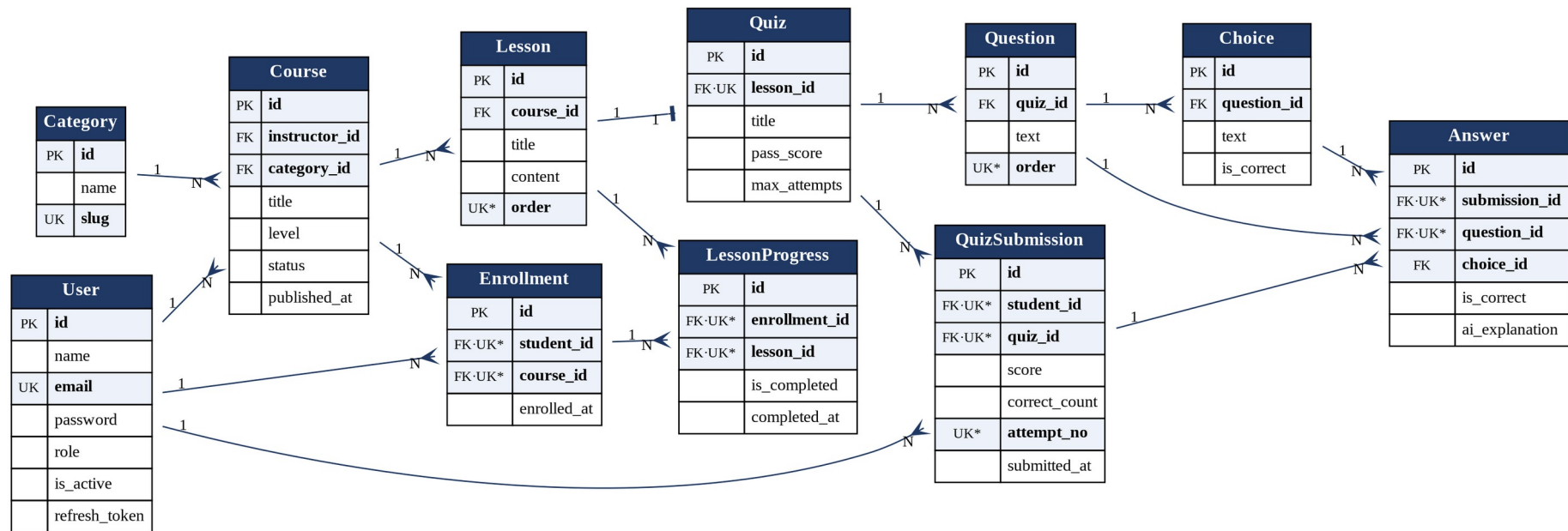
```
// Thành công
{ "success": true, "message": "...", "data": { ... },
  "meta": { "page": 1, "limit": 12, "total": 42 } }

// Thất bại
{ "success": false, "message": "Mô tả lỗi bằng tiếng Việt",
  "errors": { "email": "Email không đúng định dạng" } }
```

3.4. Thiết kế cơ sở dữ liệu

3.4.1. Sơ đồ quan hệ thực thể

Cơ sở dữ liệu gồm 11 bảng và ba kiểu liệt kê. Sơ đồ quan hệ thực thể được trình bày ở Hình 3.3 (trang sau, khổ ngang). Ký hiệu **PK** là khóa chính, **FK** là khóa ngoại, **UK** là ràng buộc duy nhất, **UK*** là thành phần của một ràng buộc duy nhất phức hợp.



Hình 3.3. Sơ đồ quan hệ thực thể của hệ thống LearnQuiz

3.4.2. Từ điển dữ liệu

Bảy bảng cốt lõi được mô tả chi tiết dưới đây; bốn bảng còn lại có cấu trúc đơn giản và đã thể hiện đầy đủ trên sơ đồ. Toàn văn lược đồ được đưa vào Phụ lục B.

Bảng 3.7. Bảng users — Người dùng

Cột	Kiểu	Ràng buộc	Diễn giải
id	SERIAL	PK	Khóa chính tự tăng
name	VARCHAR(100)	NOT NULL	Họ tên hiển thị
email	VARCHAR(150)	UNIQUE	Dùng để đăng nhập
password	TEXT	NOT NULL	Chuỗi băm bcrypt 10 vòng muối, không bao giờ trả về máy khách
avatar	VARCHAR(500)	NULL	Địa chỉ ảnh đại diện
role	ENUM Role	NOT NULL	student · instructor · admin, mặc định student
is_active	BOOLEAN	NOT NULL	Quản trị viên khóa tài khoản bằng cờ này
refresh_token	TEXT	NULL	Mã làm mới phiên đang hiệu lực; xóa đi là thu hồi phiên ngay
created_at	TIMESTAMP	NOT NULL	Thời điểm tạo
updated_at	TIMESTAMP	NOT NULL	Thời điểm cập nhật gần nhất

Bảng 3.8. Bảng courses — Khóa học

Cột	Kiểu	Ràng buộc	Diễn giải
id	SERIAL	PK	Khóa chính
instructor_id	INT	FK → users	Giảng viên sở hữu; xóa theo dây chuyền
category_id	INT	FK → categories, NULL	Xóa danh mục thì đặt về NULL, không xóa khóa học
title	VARCHAR(200)	NOT NULL	Tên khóa học
description	TEXT	NULL	Mô tả
thumbnail	VARCHAR(500)	NULL	Ảnh bìa
level	ENUM CourseLevel	NOT NULL	beginner · intermediate · advanced

Cột	Kiểu	Ràng buộc	Diễn giải
status	ENUM CourseStatus	NOT NULL	draft · pending · published · rejected
reject_reason	TEXT	NULL	Lý do quản trị viên từ chối
published_at	TIMESTAMP	NULL	Mốc thời gian được duyệt công khai

Bảng này có hai chỉ mục phụ trên status và category_id — hai cột được dùng trong hầu hết truy vấn lọc ở trang danh sách khóa học.

Bảng 3.9. Bảng lessons — Bài học

Cột	Kiểu	Ràng buộc	Diễn giải
id	SERIAL	PK	Khóa chính
course_id	INT	FK → courses	Khóa học chứa bài này
title	VARCHAR(200)	NOT NULL	Tiêu đề bài học — hiển thị công khai cho mọi người
content	TEXT	NULL	Nội dung đầy đủ — chỉ trả về cho người đã ghi danh
video_url	VARCHAR(500)	NULL	Địa chỉ video ngoài; hệ thống không tự lưu trữ video
order	INT	UNIQUE(course_id, order)	Thứ tự trong khóa; ràng buộc chặn hai bài trùng số thứ tự

Bảng 3.10. Bảng quizzes — Bài kiểm tra

Cột	Kiểu	Ràng buộc	Diễn giải
id	SERIAL	PK	Khóa chính
lesson_id	INT	FK → lessons, UNIQUE	Ràng buộc duy nhất tạo nên quan hệ một-một với bài học
title	VARCHAR(200)	NOT NULL	Tên bài kiểm tra
pass_score	INT	NOT NULL	Ngưỡng đạt tính theo thang 100, mặc định 70
max_attempts	INT	NULL	Số lượt làm tối đa; NULL nghĩa là không giới hạn

Bảng 3.11. Bảng choices — Đáp án

Cột	Kiểu	Ràng buộc	Diễn giải
id	SERIAL	PK	Khóa chính

Cột	Kiểu	Ràng buộc	Diễn giải
question_id	INT	FK → questions	Câu hỏi chứa đáp án này
text	TEXT	NOT NULL	Nội dung đáp án
is_correct	BOOLEAN	NOT NULL	Cột nhạy cảm nhất của hệ thống — không được phép xuất hiện trong phản hồi gửi cho học viên trước khi nộp bài

Bảng 3.12. Bảng quiz_submissions — Lượt nộp bài

Cột	Kiểu	Ràng buộc	Diễn giải
id	SERIAL	PK	Khóa chính
student_id	INT	FK → users	Học viên nộp bài
quiz_id	INT	FK → quizzes	Bài kiểm tra
score	INT	NOT NULL	Điểm thang 0–100 do máy chủ tính
correct_count	INT	NOT NULL	Số câu đúng
total_questions	INT	NOT NULL	Tổng số câu của đề tại thời điểm nộp
attempt_no	INT	UNIQUE(student_id, quiz_id, attempt_no)	Lượt làm thứ mấy; ràng buộc chặn nộp trùng cùng một lượt
submitted_at	TIMESTAMP	NOT NULL	Thời điểm nộp

Bảng 3.13. Bảng answers — Câu trả lời

Cột	Kiểu	Ràng buộc	Diễn giải
id	SERIAL	PK	Khóa chính
submission_id	INT	FK → quiz_submissions	Thuộc lượt nộp nào
question_id	INT	UNIQUE(submission_id, question_id)	Mỗi câu hỏi chỉ có một câu trả lời trong một lượt nộp
choice_id	INT	FK → choices, NULL	Đáp án đã chọn; NULL nghĩa là bỏ trống
is_correct	BOOLEAN	NOT NULL	Ảnh chụp kết quả chấm tại thời điểm nộp
ai_explanation	TEXT	NULL	Lời giải thích do Gemini sinh; lưu lại để không phải gọi AI lần thứ hai

3.4.3. Sáu quyết định thiết kế cần giải trình

(1) Đặt ràng buộc duy nhất trên `quizzes.lesson_id` để tạo quan hệ một–một. Đề tài quy định mỗi bài học có tối đa một quiz. Nếu chỉ kiểm tra bằng câu lệnh điều kiện trong mã nguồn, hai yêu cầu đến gần như đồng thời vẫn có thể tạo ra hai bản ghi: cả hai cùng đọc thấy “chưa có quiz” rồi cùng ghi. Ràng buộc duy nhất đẩy việc bảo đảm này xuống tầng cơ sở dữ liệu — nơi duy nhất không thể bị vượt qua.

(2) Ràng buộc duy nhất phức hợp `(student_id, course_id)` trên bảng ghi danh. Người dùng bấm nút *Ghi danh* hai lần thật nhanh sẽ tạo hai bản ghi nếu chỉ dùng cặp thao tác “tìm rồi tạo”. Với ràng buộc này, lần ghi thứ hai làm Prisma ném lỗi P2002, được bộ xử lý lỗi chuyển thành mã 409 kèm thông báo tiếng Việt.

(3) Ràng buộc duy nhất phức hợp `(course_id, order)` trên bảng bài học. Đề tài nhấn mạnh *thứ tự bài học quan trọng*, nên thứ tự là dữ liệu chứ không phải quy ước trình bày. Hệ quả phải chấp nhận: khi đổi chỗ hai bài học, việc cập nhật phải nằm trong một giao dịch và đi qua giá trị trung gian, nếu không sẽ vi phạm ràng buộc ở bước giữa. Đây là đánh đổi có chủ đích — chấp nhận phức tạp hơn khi ghi để dữ liệu không bao giờ sai.

(4) Bảng câu trả lời lưu đáp án đã chọn, đồng thời lưu kết quả chấm. Nếu về sau giảng viên sửa đáp án đúng của một câu hỏi, những bài đã nộp trước đó vẫn giữ nguyên lựa chọn thật của học viên, còn cột `is_correct` là **ảnh chụp kết quả tại thời điểm chấm**. Dữ liệu lịch sử vì thế được bảo toàn, không bị viết lại theo hiện tại.

(5) Trạng thái khóa học là kiểu liệt kê bốn giá trị, không phải cờ đúng/sai. Một cờ `is_published` không diễn tả được trạng thái “*đã gửi, đang chờ duyệt*” — vốn chính là yêu cầu trọng tâm của vai trò quản trị viên. Bốn trạng thái cho phép mô hình hóa đầy đủ vòng đời, kể cả nhánh bị từ chối rồi sửa và gửi lại.

(6) Không lưu sẵn phần trăm tiến độ. Tiến độ được tính khi cần, bằng tỉ lệ giữa số bài đã đánh dấu hoàn thành và tổng số bài của khóa. Nếu lưu sẵn một cột `progress_percent`, mỗi lần giảng viên thêm bài học mới, toàn bộ giá trị đã lưu của mọi học viên đều trở nên sai.

3.4.4. Tổng hợp ràng buộc toàn vẹn

Bảng 3.14. Ràng buộc toàn vẹn đặt tại tầng cơ sở dữ liệu

Bảng	Ràng buộc	Ngăn chặn tình huống
users	UNIQUE(email)	Hai tài khoản cùng địa chỉ thư điện tử
categories	UNIQUE(slug)	Hai danh mục cùng đường dẫn thân thiện
lessons	UNIQUE(course_id, order)	Hai bài học cùng số thứ tự trong một khóa
quizzes	UNIQUE(lesson_id)	Một bài học có hơn một bài kiểm tra
questions	UNIQUE(quiz_id, order)	Hai câu hỏi cùng số thứ tự trong một đề
enrollments	UNIQUE(student_id, course_id)	Ghi danh trùng khi bấm nút nhiều lần
lesson_progress	UNIQUE(enrollment_id, lesson_id)	Đánh dấu hoàn thành trùng lặp
quiz_submissions	UNIQUE(student_id, quiz_id, attempt_no)	Nộp trùng cùng một lượt làm bài
answers	UNIQUE(submission_id, question_id)	Một câu hỏi có hai câu trả lời trong cùng lượt nộp

Về hành vi xóa dây chuyền: xóa khóa học sẽ xóa theo toàn bộ bài học, quiz, câu hỏi và đáp án thuộc về nó; nhưng xóa một danh mục chỉ đặt category_id của khóa học về NULL, vì khóa học có giá trị độc lập với danh mục. Xóa một đáp án cũng chỉ đặt choice_id trong bảng câu trả lời về NULL, để lịch sử làm bài không biến mất.

3.5. Thiết kế giao diện lập trình

3.5.1. Quy ước chung

Toàn bộ API nằm dưới /api/v1. Cột **Quyền** trong các bảng dưới đây dùng quy ước: dấu gạch ngang là công khai, ổ khóa là cần đăng nhập, **S** là học viên, **I** là giảng viên, **A** là quản trị viên. Nhiều điểm cuối còn kiểm tra thêm quyền sở hữu tài nguyên ngoài vai trò.

3.5.2. Đặc tả các nhóm điểm cuối

Bảng 3.15. Nhóm xác thực /auth

Phương thức	Điểm cuối	Quyền	Mô tả
POST	/auth/register	—	Đăng ký; chỉ chấp nhận vai trò student hoặc instructor
POST	/auth/login	—	Trả về thông tin tài khoản kèm access token 15 phút và refresh token 7 ngày
POST	/auth/refresh	—	Cấp cập token mới sau khi đối chiếu với bản lưu trong cơ sở dữ liệu
POST	/auth/logout		Xóa mã làm mới trong cơ sở dữ liệu — phiên mất hiệu lực ngay
GET	/auth/me		Thông tin tài khoản hiện tại, không bao giờ kèm mật khẩu

Bảng 3.16. Nhóm danh mục và khóa học

Phương thức	Điểm cuối	Quyền	Mô tả
GET	/categories	—	Danh sách danh mục kèm số khóa học mỗi danh mục
POST · PATCH · DELETE	/categories	A	Quản lý danh mục; chặn 409 nếu danh mục còn khóa học
GET	/courses	—	Lọc theo danh mục, độ khó, từ khóa; sắp xếp và phân trang; chỉ trả khóa đã duyệt
GET	/courses/mine	I · A	Khóa học của tôi ở mọi trạng thái
GET	/courses/:id	—	Chi tiết kèm danh sách tiêu đề bài học; khóa chưa duyệt chỉ chủ sở hữu và quản trị viên xem được
POST	/courses	I · A	Tạo khóa học mới, trạng thái bị ép về draft
PATCH	/courses/:id	I (chủ sở hữu) · A	Sửa nội dung; không đổi được trạng thái qua điểm cuối này
POST	/courses/:id/submit	I (chủ sở hữu) · A	Gửi duyệt; chặn nếu khóa học chưa có bài học nào

Phương thức	Điểm cuối	Quyền	Mô tả
DELETE	/courses/:id	I (chủ sở hữu) · A	Chặn 409 nếu đã có học viên ghi danh

Bảng 3.17. Nhóm bài học, ghi danh và tiến độ

Phương thức	Điểm cuối	Quyền	Mô tả
GET	/courses/:id/lessons	I (chủ sở hữu) · A	Bài học kèm nội dung đầy đủ — dùng cho màn hình soạn bài
POST	/courses/:id/lessons	I (chủ sở hữu) · A	Thêm bài học; bỏ trống thứ tự thì tự xếp vào cuối
PATCH	/courses/:id/lessons/reorder	I (chủ sở hữu) · A	Sắp xếp lại toàn bộ thứ tự — giao dịch hai pha
PATCH · DELETE	/lessons/:id	I (chủ sở hữu) · A	Sửa hoặc xóa bài học
POST	/courses/:id/enroll		Ghi danh; chặn khóa chưa duyệt và chặn chính giảng viên của khóa
GET	/enrollments/me		Khóa đã ghi danh kèm số bài đã xong và phần trăm tiến độ
GET	/courses/:id/learn	 đã ghi danh	Một lời gọi trả đủ: khóa học, danh sách bài, trạng thái hoàn thành và mở khóa, tiến độ
GET	/lessons/:id	 đã ghi danh	Nội dung đầy đủ; chặn nếu bài chưa được mở khóa
PATCH	/lessons/:id/complete	 đã ghi danh	Đánh dấu hoặc bỏ đánh dấu hoàn thành, trả về tiến độ mới

Bảng 3.18. Nhóm bài kiểm tra

Phương thức	Điểm cuối	Quyền	Mô tả
GET	/lessons/:id/quiz	 đã ghi danh	Lấy đề — tuyệt đối không trả cờ đáp án đúng ; chặn nếu bài chưa mở khóa
GET	/lessons/:id/quiz/editor	I (chủ sở hữu) · A	Đề kèm đáp án đúng, dùng cho màn hình soạn quiz
PUT	/lessons/:id/quiz	I (chủ sở hữu) · A	Tạo hoặc thay thế trọn gói bộ câu hỏi trong một giao dịch; chặn 409 nếu đã có lượt nộp
PATCH	/quiz/:id	I (chủ sở hữu) · A	Sửa tên, ngưỡng đạt, số lượt — luôn cho phép

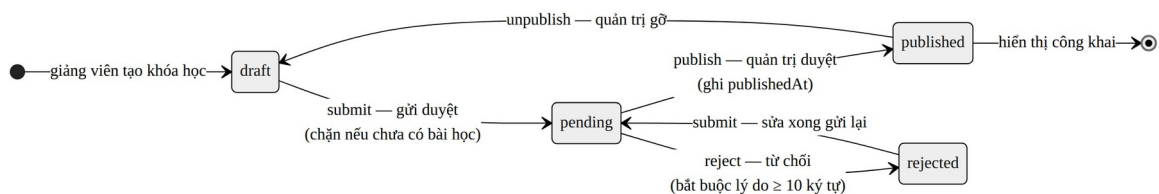
Phương thức	Điểm cuối	Quyền	Mô tả
DELETE	/quiz/:id	I (chủ sở hữu) · A	Xóa bài kiểm tra cùng toàn bộ kết quả cũ
POST	/quiz/:id/submit	 đã ghi danh	Nộp bài; máy chủ chấm điểm rồi trả kết quả kèm đáp án đúng
GET	/quiz/:id/submissions/me		Lịch sử các lượt làm bài của chính mình
GET	/submissions/:id	 chủ nhân · I · A	Chi tiết bài đã nộp: từng câu đúng hay sai

Bảng 3.19. Nhóm quản trị, thống kê và trí tuệ nhân tạo

Phương thức	Điểm cuối	Quyền	Mô tả
GET	/admin/courses	A	Hàng đợi duyệt kèm số đếm theo trạng thái; khóa chờ lâu nhất xếp đầu
PATCH	/courses/:id/publish	A	Chuyển <i>Chờ duyệt</i> thành <i>Công khai</i> , ghi mốc thời gian
PATCH	/courses/:id/reject	A	Từ chối — bắt buộc lý do từ 10 ký tự trở lên
PATCH	/courses/:id/unpublish	A	Gỡ khóa học; học viên đã ghi danh vẫn học tiếp được
GET	/users	A	Danh sách người dùng — không bao giờ trả mật khẩu hay mã làm mới phiên
PATCH	/users/:id/status	A	Khóa hoặc mở tài khoản; khóa thì thu hồi mã làm mới phiên
GET	/courses/:id/stats	I (chủ sở hữu) · A	Sĩ số, tiến độ trung bình, điểm trung bình và tỉ lệ đạt từng bài kiểm tra
GET	/admin/stats	A	Tổng quan toàn hệ thống và top 5 khóa học đông học viên nhất
GET	/ai/status		Máy chủ đã cấu hình khóa Gemini hay chưa — giao diện dùng để bật hoặc tắt các nút AI
POST	/ai/lessons/:id/generate-quiz	I (chủ sở hữu) · A	Sinh bản nháp 1–10 câu hỏi; không tự ghi vào cơ sở dữ liệu
POST	/ai/explain-answer	 chủ nhân	Giải thích vì sao đáp án đã chọn là sai; lưu lại để lần sau không gọi AI nữa

Phương thức	Điểm cuối	Quyền	Mô tả
POST	/ai/lessons/:id/summarize	🔒 đã ghi danh	Tóm tắt bài học thành 3–5 ý

3.6. Thiết kế vòng đời khóa học



Hình 3.4. Máy trạng thái vòng đời khóa học

Toàn bộ luật chuyển trạng thái được gom vào một bảng khai báo duy nhất, dùng chung cho cả tầng nghiệp vụ của giảng viên lẫn của quản trị viên. Cách làm này loại trừ khả năng hai nơi trong hệ thống hiểu luật khác nhau. Ma trận dưới đây liệt kê đủ 16 tổ hợp trạng thái × thao tác và tất cả đều được kiểm thử tự động.

Bảng 3.20. Ma trận chuyển trạng thái (16 tổ hợp)

Trạng thái hiện tại	submit	publish	reject	unpublish
draft — Bản nháp	→ pending	409	409	409
pending — Chờ duyệt	409	→ published	→ rejected	409
published — Công khai	409	409	409	→ draft
rejected — Bị từ chối	→ pending	409	409	409

Mọi ô ghi 409 đều trả về thông báo giải thích bằng tiếng Việt, nêu rõ trạng thái hiện tại và những trạng thái mà thao tác đó cho phép. Ngoài ma trận này còn một luật độc lập với trạng thái: **giảng viên không bao giờ được duyệt khóa học của chính mình**, kể cả khi là chủ sở hữu — trường hợp đó trả về mã 403.

3.7. Thiết kế giao diện người dùng

Hệ thống gồm 21 màn hình, chia theo vai trò. Mỗi màn hình được bảo vệ bằng thành phần định tuyến có kiểm tra vai trò ở phía trình duyệt, nhưng lớp bảo vệ thật vẫn nằm ở máy chủ — ấn nút bằng CSS không phải là bảo mật.

Bảng 3.21. Danh sách màn hình theo vai trò

Nhóm	Màn hình
Công khai	Trang chủ · Danh sách khóa học · Chi tiết khóa học · Đăng nhập · Đăng ký · Trang 404 · Trang 403
Học viên	Bảng điều khiển · Khóa học của tôi · Màn hình học bài · Màn hình làm quiz · Màn hình kết quả quiz
Giảng viên	Khóa học của tôi · Biểu mẫu khóa học · Trình soạn bài học · Trình soạn quiz · Thống kê lớp học
Quản trị viên	Tổng quan hệ thống · Hàng đợi duyệt khóa học · Quản lý người dùng · Quản lý danh mục

Ba nguyên tắc trình bày được áp dụng nhất quán. **Một là phản hồi ngay:** mọi thao tác ghi đều hiển thị trạng thái đang xử lý và thông báo kết quả. **Hai là không bao giờ để người dùng lạc đường:** mọi màn hình con đều có nút quay lại đúng ngữ cảnh cha. **Ba là nội dung do AI sinh luôn được ghi chú rõ** kèm câu nhắc “*Nội dung do AI đề xuất, cần kiểm tra lại với bài học*” — người học phải biết đâu là nội dung của giảng viên, đâu là gợi ý của máy.

CHƯƠNG 4. CÀI ĐẶT HỆ THỐNG

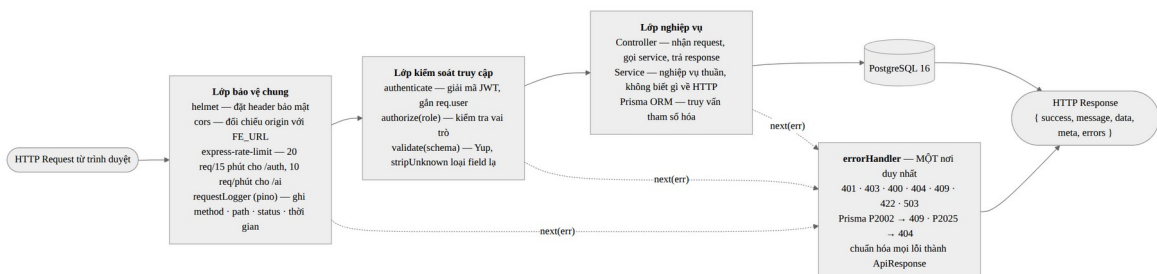
Chương này trình bày cách hiện thực hóa thiết kế ở Chương 3. Thay vì liệt kê toàn bộ mã nguồn, chương tập trung vào những đoạn mã và quyết định kỹ thuật quyết định tính đúng đắn, tính an toàn và khả năng bảo trì của hệ thống.

4.1. Tổ chức mã nguồn

Dự án được tổ chức theo mô hình một kho mã nguồn chứa hai dự án con (*monorepo*). Cách này giữ cho hai đầu luôn cùng phiên bản, và cho phép một quy trình tích hợp liên tục duy nhất kiểm tra cả hai.

```
FinalProject/
├── docker-compose.yml           # PostgreSQL 16 + Adminer khi phát triển
├── docker-compose.full.yml      # chạy trọn bộ ba thành phần bằng Docker
├── render.yaml                 # bản thiết kế hạ tầng cho Render
├── .github/workflows/ci.yml     # tích hợp liên tục
├── backend/
│   ├── prisma/schema.prisma     # 11 bảng, 3 kiểu liệt kê – nguồn sự thật
│   ├── prisma/seed.ts           # 5 tài khoản, 4 danh mục, 3 khóa học, 5 quiz
│   ├── src/config/env.ts        # đọc, kiểm tra biến môi trường khi khởi động
│   ├── src/middleware/          # xác thực · phân quyền · kiểm chứng · lỗi
│   ├── src/schemas/            # quy tắc dữ liệu bằng Yup
│   ├── src/services/            # nghiệp vụ; 3 mô-đun hàm thuần tách riêng
│   ├── src/controllers/        # nhận request, gọi service, trả response
│   ├── src/routes/              # khai báo điểm cuối và chuỗi middleware
│   ├── src/app.ts · index.ts    # lắp ráp Express · khởi động và tắt an toàn
│   ├── tests/                  # 344 phép kiểm, chạy không cần cơ sở dữ liệu
│   └── Dockerfile               # dựng hai giai đoạn
├── frontend/
│   ├── vercel.json · nginx.conf # cấu hình phục vụ ứng dụng một trang
│   └── src/
│       ├── api/                # axiosClient tự làm mới token · gọi API
│       ├── context/ · hooks/    # trạng thái đăng nhập · trạng thái AI
│       ├── router/              # khai báo tuyến và ProtectedRoute theo vai trò
│       ├── components/ · pages/ # thành phần dùng lại · 21 màn hình
│       └── types/ · utils/      # kiểu dữ liệu API · xử lý lỗi
```

4.2. Vòng đời một yêu cầu HTTP



Hình 4.1. Vòng đời một yêu cầu HTTP qua chuỗi middleware

Thứ tự các mắt xích không tùy tiện. Ba nguyên tắc chi phối cách sắp xếp: **việc rẻ tiền làm trước việc đắt tiền** (giới hạn tần suất đứng trước truy vấn cơ sở dữ liệu); **việc**

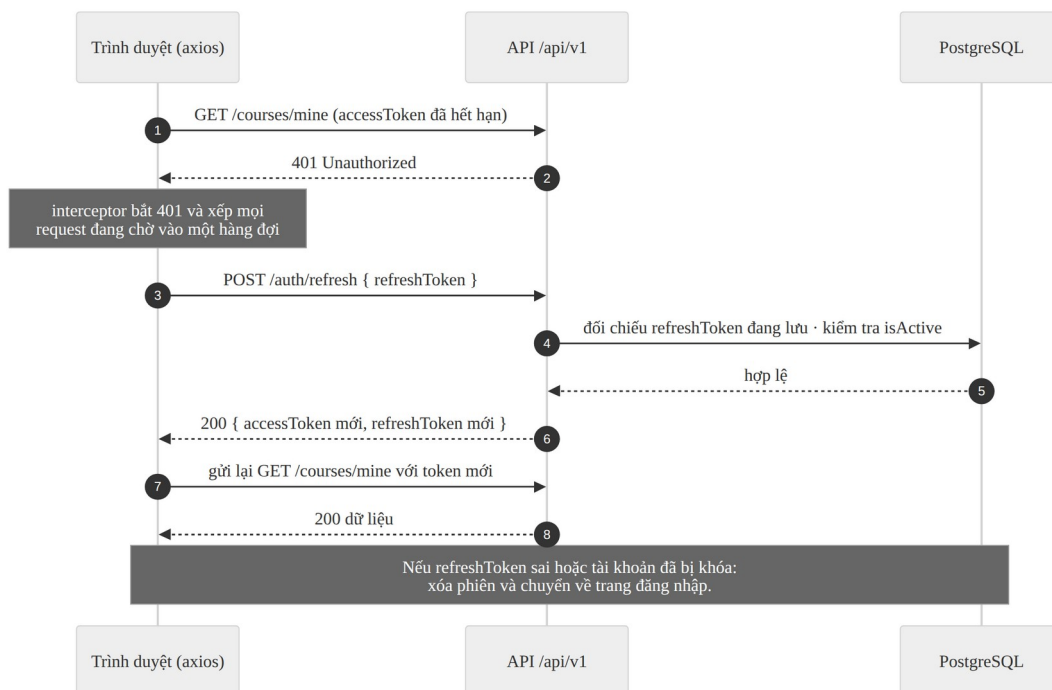
bảo vệ làm trước việc xử lý (xác thực và phân quyền đứng trước nghiệp vụ); và **mọi lỗi đổ về một chỗ** — không mắt xích nào tự định dạng phản hồi lỗi, tất cả đều gọi `next(err)` để bộ xử lý lỗi duy nhất ở cuối chuỗi chuyển thành khuôn dạng thống nhất.

Bộ xử lý lỗi cũng là nơi dịch mã lỗi của Prisma sang mã HTTP đúng ngữ nghĩa: P2002 (vi phạm ràng buộc duy nhất) thành 409, P2025 (không tìm thấy bản ghi) thành 404. Nhờ đó tầng nghiệp vụ không phải bắt lỗi cơ sở dữ liệu ở từng chỗ.

4.3. Xác thực và quản lý phiên

Khi đăng nhập thành công, máy chủ ký hai token và lưu mã làm mới vào cột `refresh_token` của người dùng. Việc lưu này biến JWT — vốn không thu hồi được — thành một phiên có thể hủy: xóa giá trị trong cơ sở dữ liệu là mọi mã làm mới cũ lập tức mất hiệu lực.

Đây cũng chính là cơ chế đứng sau hai chức năng khác: **đăng xuất** xóa mã làm mới của chính mình; **khóa tài khoản** do quản trị viên thực hiện cũng xóa mã làm mới đồng thời bật cờ `is_active = false`. Cờ này được kiểm tra ở cả bước đăng nhập lẫn bước làm mới phiên, nên tài khoản bị khóa mất quyền truy cập ngay khi access token hiện tại hết hạn.



Hình 4.2. Biểu đồ tuần tự — tự động làm mới phiên đăng nhập

Phía trình duyệt, việc làm mới phiên được đặt trong bộ chặn phản hồi của axios. Chi tiết đáng chú ý nhất là **hàng đợi**: khi một màn hình phát ra năm lời gọi song song và cả năm cùng nhận mã 401, nếu không có hàng đợi thì cả năm sẽ cùng gọi /auth/refresh, làm phát sinh năm cặp token và bốn trong số đó lập tức bị vô hiệu hóa. Cờ isRefreshing bảo đảm chỉ lời gọi đầu tiên thực sự làm mới, bốn lời gọi còn lại chờ rồi được phát lại bằng token mới.

```
let isRefreshing = false;
let queue: ((token: string) => void)[] = [];

axiosClient.interceptors.response.use(
  (response) => response,
  async (error) => {
    const original = error.config;
    if (!original || error.response?.status !== 401 || original._retry) {
      return Promise.reject(error); // không phải lỗi phiên
    }
    original._retry = true; // chỉ thử lại đúng MỘT lần

    if (isRefreshing) { // đã có lời gọi khác đang làm mới
      return new Promise((resolve) => { // xếp hàng, không gọi lại
        queue.push((token) => {
          original.headers.Authorization = `Bearer ${token}`;
          resolve(axiosClient(original));
        });
      });
    }
    // ... gọi /auth/refresh bằng axios GỐC để không lặp qua bộ chặn này
  }
);
```

4.4. Cài đặt nghiệp vụ bài kiểm tra

4.4.1. Chặn rò rỉ đáp án bằng cơ chế select

Đây là yêu cầu bảo mật quan trọng nhất của đề tài. Giải pháp không nằm ở việc “ẩn đi khi hiển thị”, mà ở chỗ **cờ đáp án đúng không bao giờ được đọc lên khỏi cơ sở dữ liệu** trong luồng lấy đề của học viên. Prisma cho phép liệt kê trường mình những cột cần lấy; những cột không liệt kê thì không có trong kết quả, do đó không thể vô tình lọt vào phản hồi:

```
const quiz = await prisma.quiz.findUnique({
  where: { lessonId },
  select: {
    id: true, title: true, passScore: true, maxAttempts: true,
    questions: {
      orderBy: { order: "asc" },
      select: {
        id: true, text: true, order: true,
        // KHÔNG có isCorrect ở đây – cố ý
        choices: { orderBy: { id: "asc" },
          select: { id: true, text: true } },
      },
    },
  },
});
```

```
| });
```

Khẳng định này được kiểm chứng tự động ở mức HTTP: bộ kiểm thử gọi thật GET /lessons/:id/quiz, lấy nguyên văn phần thân phản hồi và khẳng định chuỗi `isCorrect` không xuất hiện. Đây là kiểu kiểm thử có giá trị chứng minh cao, vì nó kiểm tra **kết quả cuối cùng gửi ra mạng** chứ không kiểm tra ý định của lập trình viên.

4.4.2. Thuật toán chấm điểm

Toàn bộ luật chấm điểm nằm trong một hàm thuần: không đụng cơ sở dữ liệu, không đụng HTTP, cùng đầu vào luôn cho cùng đầu ra. Nhờ vậy luật chấm điểm được kiểm thử độc lập và người đọc chỉ cần nhìn một chỗ là hiểu hết cách tính điểm.

```
export function gradeQuiz(questions, submitted): GradeResult {
  const answerByQuestion = new Map<number, number | null>();
  for (const answer of submitted) {
    answerByQuestion.set(answer.questionId, answer.choiceId ?? null);
  }

  // Duyệt theo DANH SÁCH CÂU HỎI THẬT trong đề, không duyệt theo bài làm
  const answers = questions.map((question) => {
    const selectedId = answerByQuestion.get(question.id) ?? null;
    // Đáp án đã chọn phải thuộc đúng câu hỏi này
    const selected = question.choices.find((c) => c.id === selectedId);
    return {
      questionId: question.id,
      choiceId: selected ? selected.id : null,
      isCorrect: Boolean(selected?.isCorrect),
    };
  });

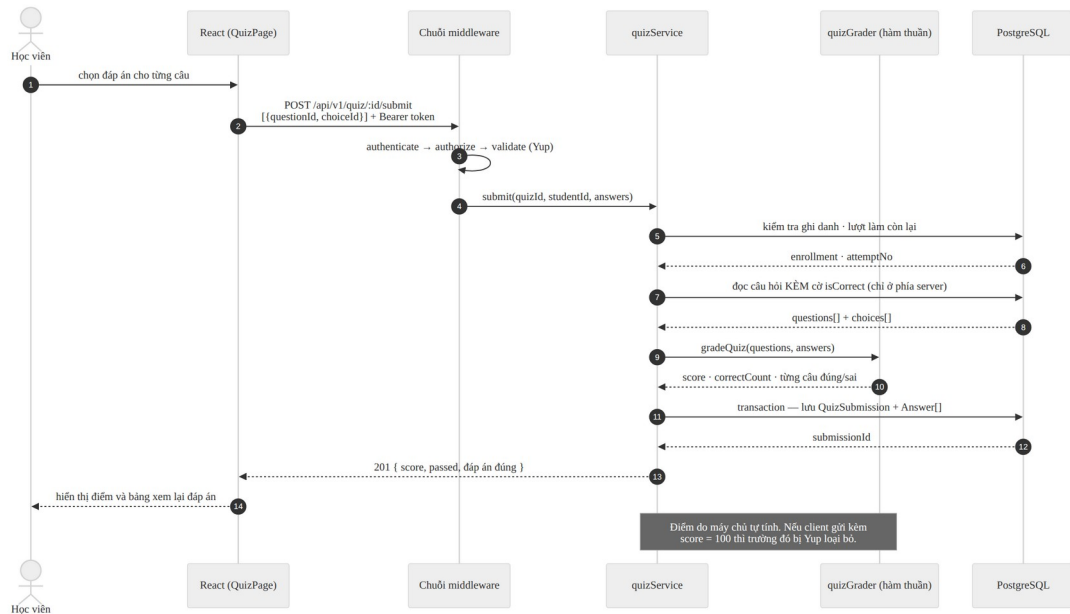
  const totalQuestions = questions.length;
  const correctCount = answers.filter((a) => a.isCorrect).length;
  return {
    totalQuestions, correctCount, answers,
    score: totalQuestions === 0 ? 0
      : Math.round((correctCount / totalQuestions) * 100),
  };
}
```

Hai chi tiết nhỏ nhưng chặn được hai cách gian lận thật sự:

- **Duyệt theo đề, không duyệt theo bài làm.** Nếu vòng lặp chạy trên danh sách câu trả lời gửi lên, học viên chỉ cần bỏ trống ba câu trong đề năm câu là mẫu số còn hai — trả lời đúng cả hai sẽ được 100 điểm. Duyệt theo đề khiến câu bỏ trống vẫn nằm trong mẫu số.
- **Đáp án phải thuộc đúng câu hỏi.** Gửi mã đáp án của câu khác — hoặc một số bịa ra — đều bị coi như bỏ trống. Nếu chỉ tra cứu mã đáp án trong toàn bộ bảng, một mã đáp án đúng của câu khác sẽ được tính điểm.

Trường hợp đề rồng trả về 0 điểm thay vì thực hiện phép chia cho 0, tránh giá trị không xác định lọt vào cơ sở dữ liệu.

4.4.3. Luồng nộp bài



Hình 4.3. Biểu đồ tuần tự — nộp bài quiz và chấm điểm

Việc lưu lượt nộp và toàn bộ câu trả lời được đặt trong **một giao dịch**: hoặc lưu trọn vẹn, hoặc không lưu gì. Không thể xảy ra tình trạng có bản ghi điểm mà thiếu chi tiết câu trả lời.

Số lượt làm bài được tính từ dữ liệu (attemptNo lớn nhất đã có, cộng một) chứ không do máy khách gửi lên, và ràng buộc duy nhất (student_id, quiz_id, attempt_no) chặn nốt trường hợp hai yêu cầu đến đồng thời.

4.4.4. Khóa sửa đề sau khi đã có bài nộp

Điểm cuối thay thế trọn gói bộ câu hỏi trả về mã 409 nếu bài kiểm tra đã có ít nhất một lượt nộp. Lý do: nếu cho phép thay đề, những bài đã chấm sẽ tham chiếu tới các câu hỏi không còn tồn tại, và điểm cũ trở nên vô nghĩa. Ngược lại, việc sửa tên bài kiểm tra, ngưỡng đạt hay số lượt tối đa **luôn được phép**, vì các thuộc tính này không làm hỏng dữ liệu lịch sử.

4.5. Lộ trình học tuần tự

Luật mở khóa được tách thành mô-đun riêng, cố tình **không nhập khẩu Prisma**, nhờ đó tệp kiểm thử nạp được mà không kéo theo toàn bộ trình điều khiển cơ sở dữ liệu:

```
export function computeUnlock(lessons, completedIds, bypass) {
  const sorted = [...lessons].sort((a, b) => a.order - b.order);
  const unlocked = new Map<number, boolean>();

  let allPreviousCompleted = true;
  for (const lesson of sorted) {
    unlocked.set(lesson.id, bypass || allPreviousCompleted);
    allPreviousCompleted =
      allPreviousCompleted && completedIds.has(lesson.id);
  }
  return unlocked;
}
```

Bản cài đặt đầu tiên chỉ xét **bài liền trước**. Chính bộ kiểm thử đã phát hiện trạng thái vô lý mà cách đó tạo ra: học viên hoàn thành cả ba bài rồi bỏ đánh dấu bài 1 — bài 2 bị khóa lại trong khi bài 3 vẫn mở. Sửa thành điều kiện *tất cả các bài đứng trước đã hoàn thành* thì trạng thái luôn nhất quán. Đây là một ví dụ cụ thể cho thấy giá trị của việc tách luật nghiệp vụ thành hàm thuần: lỗi được phát hiện bằng kiểm thử tự động, trước khi bất kỳ người dùng nào gặp phải.

Hàm này được gọi ở **hai chỗ**: khi dựng danh sách bài học cho màn hình học, và khi trả nội dung một bài cụ thể. Nhờ vậy, gọi thẳng GET /lessons/:id bằng công cụ dòng lệnh cũng không vượt qua được khóa lộ trình — bảo vệ nằm ở máy chủ chứ không ở giao diện.

4.6. Máy trạng thái vòng đời khóa học

Toàn bộ luật chuyển trạng thái được khai báo trong một bảng duy nhất. Thêm một trạng thái mới chỉ phải sửa một chỗ, và mọi tổ hợp đều kiểm thử được mà không cần cơ sở dữ liệu:

```
const TRANSITIONS: Record<CourseAction, Transition> = {
  submit: { from: ["draft", "rejected"], to: "pending" },
  publish: { from: ["pending"], to: "published" },
  reject: { from: ["pending"], to: "rejected" },
  unpublish: { from: ["published"], to: "draft" },
};
```

Cùng một bảng được dùng bởi tầng nghiệp vụ của giảng viên (thao tác gửi duyệt) và của quản trị viên (duyet, từ chối, gỡ). Bảng còn sinh ra danh sách thao tác hợp lệ tại mỗi trạng thái, dùng để bật hoặc tắt nút trên giao diện — nghĩa là **giao diện và máy chủ không thể hiểu luật khác nhau**, vì cùng đọc một nguồn.

4.7. Thống kê không phát sinh truy vấn theo sĩ số

Cách viết ngây thơ khi thống kê một lớp là lặp qua từng học viên rồi truy vấn điểm của người đó — lớp 200 học viên sẽ tạo ra hơn 200 truy vấn, hiện tượng quen thuộc mang tên **N+1**. Hệ thống thay bằng các lệnh gộp nhóm chạy song song:

```
const [byQuiz, byStudent, byQuizStudent, passCounts] = await Promise.all([
  prisma.quizSubmission.groupBy({
    by: ["quizId"],
    where: { quizId: { in: quizIds } },
    _avg: { score: true }, _count: { _all: true },
  }),
  prisma.quizSubmission.groupBy({ by: ["studentId"], /* ... */ }),
  // ...
]);
```

Số truy vấn vì thế **không đổi** dù lớp có 10 hay 10.000 học viên. Đây là một trong những yêu cầu phi chức năng đã nêu ở Bảng 3.2, và cũng là điểm dễ bị hỏi khi bảo vệ.

4.8. Tích hợp trí tuệ nhân tạo

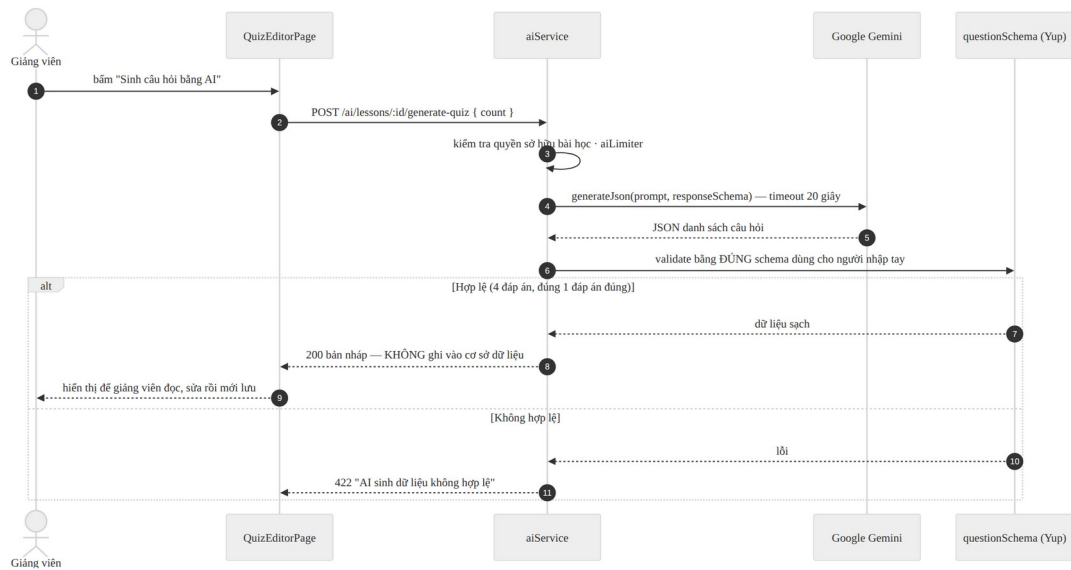
4.8.1. Lớp gọi Gemini

Lớp gọi Gemini được viết bằng hàm `fetch` sẵn có của Node.js 22, không dùng thư viện của nhà cung cấp — giảm phụ thuộc và giúp người đọc thấy rõ toàn bộ giao thức. Ba chi tiết kỹ thuật đáng nêu:

- **Khóa API đặt trong header `x-goog-api-key`**, không đặt trong chuỗi truy vấn của địa chỉ. Địa chỉ URL thường bị ghi vào nhật ký của các tầng trung gian; header thì không.
- **Có bộ hủy theo thời gian chờ 20 giây**. Không có nó, một lời gọi treo sẽ giữ kết nối cho tới khi hết thời gian mặc định của hệ điều hành, làm treo cả màn hình người dùng.
- **Buộc trả về JSON theo lược đồ**. Tham số `responseMimeType` và `responseSchema` khiến mô hình trả đúng cấu trúc mong muốn, giảm hẳn số trường hợp phải chữa cháy bằng cách phân tích văn bản tự do.

Lỗi từ dịch vụ ngoài được ánh xạ sang mã HTTP đúng ngữ nghĩa thay vì trả 500 chung chung: vượt hạn mức thành 429, sai khóa hoặc bị từ chối thành 503, quá thời gian chờ thành 504. Thông điệp chi tiết của nhà cung cấp **không** được chuyển thẳng ra ngoài, để tránh lộ thông tin cấu hình nội bộ.

4.8.2. Ba tính năng và nguyên tắc kiểm duyệt đầu ra



Hình 4.4. Biểu đồ tuần tự — sinh câu hỏi trắc nghiệm bằng Gemini

Bảng 4.1. Ba tính năng AI và ràng buộc thiết kế

Tính năng	Người dùng	Ràng buộc thiết kế
Sinh câu hỏi trắc nghiệm	Giảng viên	Chỉ trả bản nháp; không ghi vào cơ sở dữ liệu ; kết quả phải qua đúng bộ quy tắc dùng cho câu hỏi nhập tay
Giải thích đáp án sai	Học viên	Chỉ dùng cho câu đã trả lời sai của chính mình; kết quả được lưu vào <code>answers.ai_explanation</code> để lần sau không gọi lại AI
Tóm tắt bài học	Học viên đã ghi danh, giảng viên	Kết quả 3–5 ý, chỉ hiển thị, luôn kèm ghi chú nguồn gốc AI

Nguyên tắc quan trọng nhất được thể hiện gọn trong một dòng mã: đầu ra của mô hình được kiểm chứng bằng **đúng lược đồ** dùng cho dữ liệu do giảng viên gõ tay.

```
validated = await aiQuestionsSchema.validate(raw.questions, {
  abortEarly: false,
  stripUnknown: true,
});
```

Nói cách khác: **AI cũng chỉ là một nguồn đầu vào không đáng tin và không được hưởng ngoại lệ nào.** Câu hỏi do AI sinh ra mà chỉ có ba đáp án, hoặc có hai đáp án cùng được đánh dấu đúng, đều bị chặn ở mã 422 trước khi tới tay giảng viên. Hai tình huống này đều được tái hiện trong bộ kiểm thử.

4.8.3. Suy giảm êm khi không có AI

Hệ thống phải chạy được ngay cả khi chưa cấu hình khóa API — tình huống rất thực tế khi chấm bài hoặc khi hạn mức miễn phí đã hết. Điểm cuối GET /ai/status cho giao diện biết dịch vụ có sẵn sàng không; nếu không, các nút AI bị làm mờ kèm chú giải nêu rõ lý do, còn **toàn bộ chức năng còn lại hoạt động bình thường**. Bộ kiểm thử có hẳn một nhóm khẳng định điều này: thiếu khóa API thì trả mã 503 kèm hướng dẫn chứ không phải 500, và phần còn lại của hệ thống vẫn đáp ứng đúng.

Toàn bộ nhóm /ai còn đi qua một bộ giới hạn tần suất riêng (10 lượt mỗi phút cho mỗi địa chỉ), vì mỗi lời gọi tới Gemini đều phát sinh chi phí.

4.9. Cài đặt phía trình duyệt

Bảo vệ tuyến đường theo vai trò. Thành phần ProtectedRoute bọc quanh các nhánh tuyến đường, đọc trạng thái đăng nhập từ ngữ cảnh toàn cục và chuyển hướng khi không đủ quyền. Cần nhắc lại: đây là biện pháp trải nghiệm, không phải biện pháp bảo mật — mọi kiểm tra thật đều lặp lại ở máy chủ.

Biểu mẫu. React Hook Form quản lý biểu mẫu theo cơ chế không kiểm soát nên hạn chế vẽ lại; quy tắc kiểm chứng dùng chung Yup với máy chủ. Riêng các thành phần chọn của MUI phải được bọc bằng Controller thay vì đăng ký trực tiếp, do chúng không phát ra sự kiện của thẻ input gốc.

Xử lý lỗi tập trung. Một hàm duy nhất chuyển lỗi HTTP thành thông báo tiếng Việt cho người dùng, dựa trên khuôn dạng phản hồi thống nhất ở mục 3.3.3. Nhờ đó không màn hình nào phải tự đoán cấu trúc lỗi.

4.10. Bảo mật

Bảng 4.2. Rủi ro bảo mật và biện pháp đã áp dụng

Rủi ro	Biện pháp đã cài đặt
Lộ mật khẩu khi cơ sở dữ liệu bị rò rỉ	Băm bằng bcrypt 10 vòng muối; mật khẩu không bao giờ nằm trong phản hồi
Dò mật khẩu tự động	Giới hạn 20 lượt trong 15 phút cho nhóm /auth
Đánh cắp token	Access token sống 15 phút; refresh token lưu trong cơ sở dữ liệu nên thu hồi được
Dò xem địa chỉ thư có tồn tại không	Sai địa chỉ và sai mật khẩu trả về cùng một thông báo

Rủi ro	Biện pháp đã cài đặt
Gán tràn thuộc tính (gửi kèm <code>role: admin</code>)	Yup <code>stripUnknown</code> loại bỏ trường lạ; đăng ký chỉ chấp nhận <code>student</code> hoặc <code>instructor</code>
Tiêm nhiễm SQL	Prisma dùng truy vấn tham số hóa; không nối chuỗi SQL ở bất kỳ đâu
XSS và chèn khung giả mạo	<code>helmet()</code> đặt chính sách nội dung, <code>X-Frame-Options</code> , <code>X-Content-Type-Options</code>
Gọi API từ tên miền lạ	Danh sách trắng CORS đúng bằng địa chỉ front-end
Lộ khóa bí mật	Tệp <code>.env</code> nằm trong danh sách bỏ qua của git; bí mật cấu hình qua biến môi trường
Lộ dữ liệu nhạy cảm qua nhật ký	pino cấu hình che tự động các trường <code>password</code> , <code>authorization</code> , <code>token</code>
Tài khoản vi phạm vẫn dùng được	Cờ <code>is_active</code> được kiểm tra ở cả bước đăng nhập và bước làm mới phiên
Lộ khóa API của dịch vụ AI	Khóa chỉ nằm trong biến môi trường của máy chủ; gửi qua header, không qua địa chỉ URL
Mất toàn bộ quản trị viên	Không cho tự khóa mình; không cho khóa quản trị viên đang hoạt động cuối cùng

Riêng biện pháp cuối cùng đáng được giải thích thêm, vì nó gồm **hai lớp** và lớp thứ hai không hề thừa. Giả sử chỉ chặn việc tự khóa: quản trị viên B khóa quản trị viên A, nhưng access token của A còn hiệu lực thêm vài phút nên A vẫn kịp khóa ngược lại B — hệ thống mất sạch quản trị viên và không ai mở khóa được nữa. Vì vậy hệ thống chặn luôn thao tác khóa quản trị viên đang hoạt động cuối cùng. Tình huống này được tái hiện nguyên văn trong bộ kiểm thử.

4.11. Ghi nhật ký và khả năng quan sát

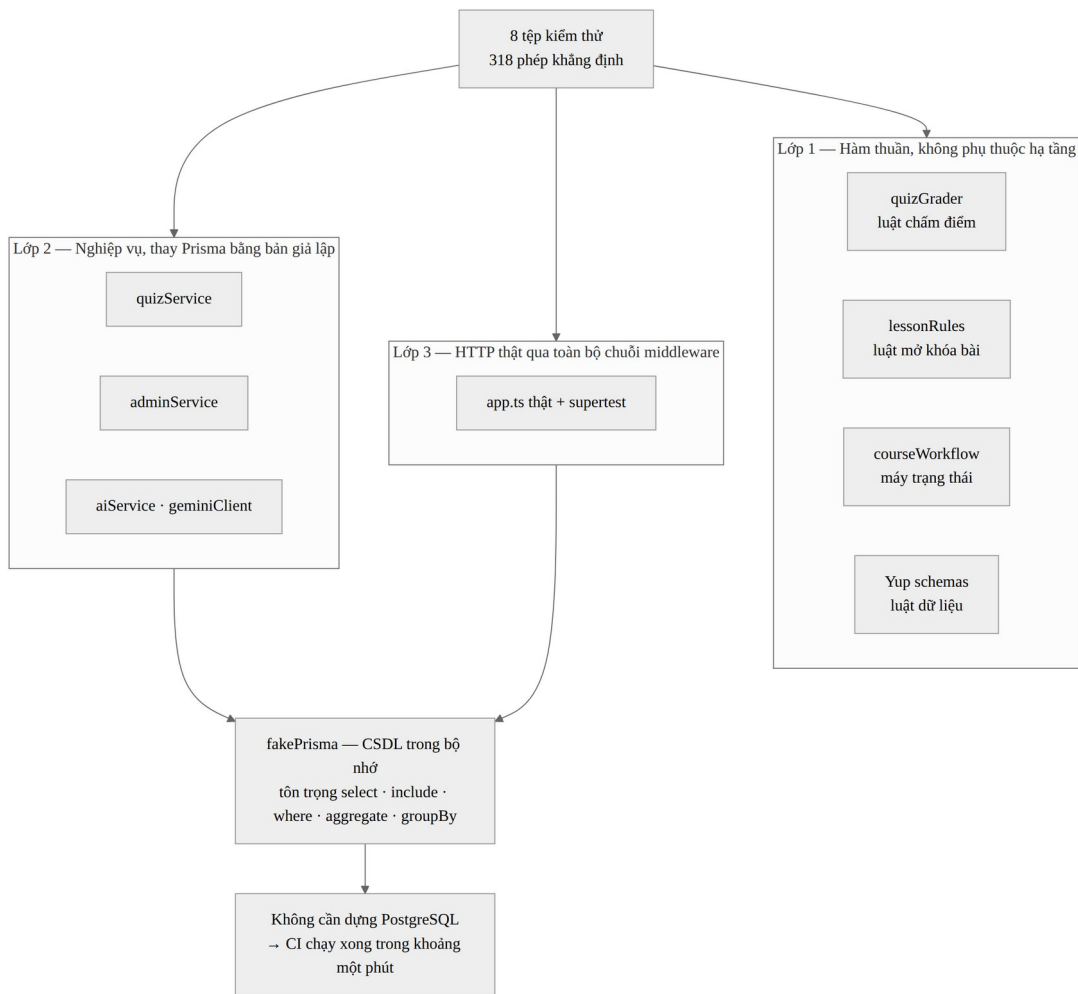
Hệ thống dùng pino ghi nhật ký dạng JSON có cấu trúc. Mỗi yêu cầu sinh ra một bản ghi gồm phương thức, đường dẫn, mã trạng thái, thời gian xử lý và địa chỉ người gọi. Cấu hình `redact` tự động che các trường nhạy cảm, nên ngay cả khi ghi nhầm cả đối tượng yêu cầu thì mật khẩu và token cũng không lọt vào nhật ký.

Điểm cuối `/health` trả về trạng thái máy chủ kèm kết quả kiểm tra kết nối cơ sở dữ liệu, dùng cho cơ chế kiểm tra sức khỏe của nền tảng triển khai và cho việc đánh thức dịch vụ trước buổi bảo vệ. Máy chủ cũng cài đặt cơ chế **tắt an toàn**: khi nhận tín hiệu dừng, tiến trình ngừng nhận yêu cầu mới, chờ các yêu cầu đang xử lý hoàn tất rồi mới đóng kết nối cơ sở dữ liệu.

CHƯƠNG 5. KIỂM THỬ VÀ TRIỂN KHAI

5.1. Chiến lược kiểm thử

Bộ kiểm thử được tổ chức theo ba lớp, đi từ trong ra ngoài. Nguyên tắc chọn lớp rất thực dụng: **kiểm thử ở lớp thấp nhất còn chứng minh được điều cần chứng minh**, vì lớp càng thấp thì chạy càng nhanh và lỗi càng dễ định vị.



Hình 5.1. Kiến trúc ba lớp của bộ kiểm thử

- **Lớp 1 — hàm thuần.** Luật chấm điểm, luật mở khóa bài học, máy trạng thái khóa học và các lược đồ kiểm chứng. Không cần cơ sở dữ liệu, không cần máy chủ; chạy trong vài chục mili giây.
- **Lớp 2 — nghiệp vụ.** Gọi thẳng các hàm nghiệp vụ với tầng Prisma được thay bằng bản giả lập trong bộ nhớ. Kiểm tra phân quyền, quy tắc nghiệp vụ và số liệu thống kê.

- **Lớp 3 — HTTP thật.** Nạp đúng ứng dụng Express thật và gửi yêu cầu HTTP qua toàn bộ chuỗi middleware. Đây là lớp duy nhất chứng minh được những khẳng định về **nội dung thật sự gửi ra mạng**.

5.2. Prisma giả lập trong bộ nhớ

Trở ngại lớn nhất khi kiểm thử tầng nghiệp vụ là sự phụ thuộc vào cơ sở dữ liệu. Dựng PostgreSQL trong môi trường tích hợp liên tục vừa chậm vừa dễ hỏng. Giải pháp là một bản giả lập Prisma khoảng 380 dòng, chạy hoàn toàn trong bộ nhớ.

Điểm mấu chốt khiến bản giả lập này có giá trị chứng minh: **nó tôn trọng `select` và `include`**. Một bản giả lập ngây thơ luôn trả về đầy đủ mọi cột sẽ khiến phép thử “không rò rỉ đáp án” trở nên vô nghĩa — phép thử sẽ thất bại ngay cả khi mã nguồn thật hoàn toàn đúng, hoặc tệ hơn, sẽ đạt vì lý do sai. Bản giả lập còn hỗ trợ các toán tử điều kiện (in, gte, contains, OR, lọc theo quan hệ), các phép tổng hợp, gộp nhóm, sắp xếp theo số lượng bản ghi quan hệ và ghi lồng nhiều mức.

Trong quá trình thực hiện, hai khiếm khuyết của chính bản giả lập đã bị phát hiện: chưa hỗ trợ ghi lồng hai mức, và thiếu một số quan hệ. Cả hai đều là lỗi của công cụ kiểm thử chứ không phải của sản phẩm — nhưng việc phân biệt được hai loại lỗi này cũng là một phần của kỹ năng kiểm thử.

5.3. Kết quả kiểm thử tự động

Toàn bộ bộ kiểm thử chạy bằng một lệnh duy nhất và **không cần cơ sở dữ liệu**. Kết quả lần chạy gần nhất: **344/344 phép khẳng định đạt**, mã thoát 0.

Bảng 5.1. Kết quả bộ kiểm thử tự động

Tập kiểm thử	Nội dung kiểm	Số phép	Kết quả
env.test.ts	Model Gemini mặc định không bị cấu hình cục bộ làm sai lệch	1	Đạt
grader.test.ts	Luật chấm điểm và luật mở khóa bài học	24	Đạt
workflow.test.ts	Máy trạng thái khóa học — đủ 16 tổ hợp	30	Đạt
schema.test.ts	Luật ra đề, luật nộp bài, bộ lọc quản trị, chống gán tràn thuộc tính	31	Đạt
gemini.test.ts	Lớp gọi Gemini: JSON, lỗi mạng, quá thời gian, không lộ chi tiết lỗi	18	Đạt

Tập kiểm thử	Nội dung kiểm	Số phép	Kết quả
quizService.test.ts	Nghiệp vụ quiz đầu-cuối: phân quyền, chấm điểm, giới hạn lượt, khóa sửa đề, chặn làm lại sau khi đã đạt	49	Đạt
adminService.test.ts	Duyệt khóa học, quản lý người dùng, toàn bộ số liệu thống kê	75	Đạt
aiService.test.ts	Ba tính năng AI: phân quyền, kiểm duyệt đầu ra, bộ nhớ đệm	36	Đạt
enrollmentService.test.ts	Điểm quiz trung bình theo khóa học, lấy lượt có điểm cao nhất	4	Đạt
graduationRegression.test.ts	Hồi quy theo review tốt nghiệp: khóa lộ trình, đóng vòng duyệt nội dung, chặn hard-delete có dữ liệu học tập, bất biến còn ít nhất 1 admin, snapshot điểm đạt	16	Đạt
api.test.ts	Bảng định tuyến thật và gọi HTTP qua toàn bộ chuỗi middleware	60	Đạt
Tổng cộng		344	344/344

Bảy khẳng định dưới đây là những phép thử có giá trị chứng minh cao nhất — cũng là những điều nên trình diễn trực tiếp khi bảo vệ.

Bảng 5.2. Bảy khẳng định quan trọng nhất mà bộ kiểm thử bảo vệ

#	Khẳng định
1	Phản hồi HTTP thật của GET /lessons/:id/quiz không chứa chuỗi `isCorrect`
2	Gửi kèm score: 100 khi nộp bài thì máy chủ vẫn tự chấm ra điểm thật (0 điểm nếu sai hết)
3	Chọn mã đáp án của câu hỏi khác thì không được tính điểm , được ghi nhận là bỏ trống
4	Phản hồi của GET /users không chứa `password` hay `refreshToken`
5	Giảng viên gọi duyệt chính khóa học của mình vẫn nhận mã 403 — không ai tự duyệt bài của mình
6	Câu hỏi do AI sinh có ba đáp án, hoặc hai đáp án đúng, đều bị chặn ở mã 422
7	Lần thứ hai xin giải thích cùng một câu không gọi lại Gemini — lấy từ bản đã lưu

5.4. Kiểm thử thủ công

Song song với kiểm thử tự động, mỗi sprint đều có một lượt kiểm thử thủ công theo kịch bản, tập trung vào những gì máy khó kiểm: trải nghiệm, thông báo lỗi và bố cục trên nhiều kích thước màn hình.

Bảng 5.3. Trích kịch bản kiểm thử thủ công

Mã	Kịch bản	Kết quả mong đợi
KT-01	Ghi danh, học bài 1, đánh dấu hoàn thành	Thanh tiến độ tăng đúng tỉ lệ; bài 2 được mở khóa
KT-02	Mở thẳng địa chỉ bài 3 khi chưa xong bài 2	Bị chặn kèm thông báo, không xem được nội dung
KT-03	Mở tab Mạng của trình duyệt khi lấy đề quiz	Phản hồi không chứa trường <code>isCorrect</code>
KT-04	Nộp bài với hai câu bỏ trống	Câu bỏ trống vẫn tính vào mẫu số; điểm đúng tỉ lệ
KT-05	Làm lại quiz quá số lượt cho phép	Nút làm lại biến mất; gọi thẳng API trả mã 409
KT-06	Giảng viên A mở màn hình sửa khóa học của giảng viên B	Trả mã 403 và chuyển sang trang báo không đủ quyền
KT-07	Quản trị viên từ chối khóa học không nhập lý do	Biểu mẫu báo lỗi; API trả mã 400 nếu gọi thẳng
KT-08	Tắt khóa API của Gemini rồi mở lại giao diện	Nút AI bị làm mờ kèm chú giải; các chức năng khác bình thường
KT-09	Đế phiên hết hạn rồi thao tác tiếp	Hệ thống tự làm mới phiên, thao tác thành công, người dùng không thấy gián đoạn
KT-10	Thu hẹp cửa sổ trình duyệt xuống kích thước điện thoại	Bố cục xếp lại theo chiều dọc, không tràn ngang

5.5. Đóng gói bằng Docker

Cả hai dự án đều có tệp Dockerfile dựng theo **hai giai đoạn**. Giai đoạn đầu cài đầy đủ thư viện và biên dịch; giai đoạn sau chỉ chép kết quả biên dịch sang một ảnh nền sạch. Ảnh cuối của máy chủ do đó không chứa mã TypeScript và không chứa thư viện chỉ dùng khi phát triển.

Hai chi tiết bảo mật và vận hành cần nêu rõ. Thứ nhất, tiến trình chạy dưới người dùng node chứ **không phải `root`** — nếu ứng dụng bị chiếm quyền, kẻ tấn công cũng không có quyền quản trị bên trong vùng chứa. Thứ hai, gói `prisma` được chuyển từ nhóm phụ thuộc phát triển sang nhóm phụ thuộc chính: lệnh cài đặt cho môi trường thật loại bỏ nhóm phát triển, mà lệnh áp dụng migration lại cần gói này — nếu để sai nhóm, vùng chứa sẽ khởi động rồi hỏng ở bước migration.

Ảnh front-end được dựng thành tệp tĩnh rồi phục vụ bằng nginx, với cấu hình chuyển hướng mọi đường dẫn về tệp `index.html` — bắt buộc phải có với ứng dụng một

trang, nếu không thì mở thẳng một địa chỉ con sẽ ra lỗi 404 vì trên đĩa không tồn tại tệp tương ứng.

5.6. Tích hợp liên tục



Hình 5.2. Quy trình tích hợp liên tục và triển khai

Bảng 5.4. Ba nhóm kiểm tra trong quy trình tích hợp liên tục

Nhóm	Các bước	Mục tiêu
backend	Kiểm tra lược đồ Prisma → kiểm tra kiểu → rà quy chuẩn mã (lint) → chạy 344 phép kiểm → biên dịch	Chặn mã hỏng vào nhánh chính
frontend	Kiểm tra kiểu → rà quy chuẩn mã (lint) → 10 unit test → dựng bản phát hành	Bảo đảm bản dựng luôn thành công
security	Chặn nếu tệp .env từng bị đưa lên git → rà lỗ hổng thư viện cả hai dự án	Ngăn rò rỉ bí mật và thư viện có lỗ hổng

Bước rà soát tệp .env quét **toàn bộ lịch sử** kho mã nguồn chứ không chỉ trạng thái hiện tại. Lý do: một tệp bí mật đã từng được đưa lên thì vẫn nằm trong lịch sử dù về sau đã xóa — và mọi thứ từng lên kho công khai đều phải coi như đã bị lộ.

Vì bộ kiểm thử dùng Prisma giả lập trong bộ nhớ, quy trình tích hợp liên tục **không cần dựng cơ sở dữ liệu**, nhờ đó hoàn tất trong khoảng một phút.

5.7. Triển khai lên môi trường thật

5.7.1. Lựa chọn hạ tầng: vì sao kết hợp Vercel và Render

Kiến trúc client–server tách rời cho phép hai đầu được triển khai độc lập, nên câu hỏi đặt ra không phải “*chọn nền tảng nào*” mà là “*giao phần nào cho nền tảng nào*”. Hệ thống chọn phương án kết hợp: **Vercel phục vụ front-end, Render chạy back-end cùng cơ sở dữ liệu PostgreSQL**. Cả hai đều dùng gói miễn phí.

Bảng 5.5. So sánh hai nền tảng theo phần việc được giao

Tiêu chí	Vercel — phục vụ front-end	Render — chạy back-end và cơ sở dữ liệu
Thành phần đảm nhiệm	frontend/ — React 19, Vite, MUI, React Router	backend/ — Express REST API và PostgreSQL
Loại dịch vụ	Lưu trữ tệp tĩnh trên mạng phân phối nội dung toàn cầu	Nền tảng cho thuê chạy tiến trình thường trú, kèm cơ sở dữ liệu do nhà cung cấp quản lý
Tốc độ tải lần đầu	Gần như tức thì — tệp được phục vụ từ điểm phát gần người dùng nhất	Phụ thuộc vị trí đặt máy chủ và trạng thái vùng chứa
Hiện tượng “ngủ đông”	Không bao giờ ngủ — tệp tĩnh luôn sẵn sàng	Ngủ sau 15 phút không có lượt truy cập ; lần gọi đầu chờ khoảng 30–50 giây
Cơ sở dữ liệu	Không có cơ sở dữ liệu quan hệ đi kèm	PostgreSQL miễn phí, kết nối nội bộ không đi ra Internet
Tự động hóa khi triển khai	Tự dựng bản phát hành, tối ưu gói tệp, cấp chứng chỉ HTTPS	Tự chạy <code>prisma migrate deploy</code> theo mô tả trong <code>render.yaml</code>
Định tuyến ứng dụng một trang	Khai báo một dòng trong <code>vercel.json</code>	Phải tự cấu hình máy chủ tệp tĩnh

Vì sao không dùng riêng Vercel? Vercel được thiết kế cho hàm phi máy chủ — mỗi lời gọi khởi tạo rồi kết thúc — chứ không cho một tiến trình Express thường trú. Quan trọng hơn, Vercel không kèm cơ sở dữ liệu quan hệ, nên vẫn phải đăng ký thêm một dịch vụ thứ ba và quản lý ba tài khoản thay vì hai.

Vì sao không dùng riêng Render? Gói miễn phí của Render cho dịch vụ web ngủ sau 15 phút không có lượt truy cập. Nếu Render phục vụ luôn cả giao diện, người chấm bấm vào đường dẫn đồ án sẽ nhìn màn hình trắng gần một phút trước khi thấy bất cứ thứ gì — một ấn tượng đầu tiên rất bất lợi mà nguyên nhân lại hoàn toàn nằm ngoài chất lượng phần mềm.

Vì sao kết hợp lại giải quyết được cả hai? Giao diện nằm trên mạng phân phối nội dung nên mở ra ngay lập tức; trong khoảng thời gian người dùng đọc trang chủ, máy chủ trên Render đã kịp thức dậy. Đến khi có thao tác đầu tiên cần gọi API — đăng nhập, xem khóa học — back-end đã sẵn sàng. Nói cách khác, độ trễ khởi động của gói miễn phí được **giấu sau thời gian đọc của người dùng** thay vì bắt họ ngồi chờ trước màn hình trắng.

5.7.2. Quy trình triển khai và ba điểm dễ sai

Máy chủ và cơ sở dữ liệu được triển khai lên Render bằng tệp mô tả hạ tầng `render.yaml`: nền tảng đọc tệp này rồi tự tạo cả cơ sở dữ liệu lẫn dịch vụ web, tự sinh các chuỗi bí mật cho JWT. Front-end được triển khai lên Vercel với thư mục gốc là `frontend/`.

Ba điểm dễ sai khi triển khai, đã được xử lý và ghi lại thành tài liệu hướng dẫn kèm theo:

- **Lệnh áp dụng migration phải nằm ở bước dừng, không nằm ở bước khởi động.** Bước dừng chạy một lần mỗi lần triển khai; bước khởi động chạy lại mỗi khi vùng chứa thức dậy — mà gói miễn phí thì ngủ và dậy liên tục.
- **Biến môi trường có tiền tố ``VITE_`` được nhúng lúc dừng, không đọc lúc chạy.** Đổi địa chỉ API xong phải dừng lại chứ không phải khởi động lại. Đây cũng là lý do tuyệt đối không đặt khóa bí mật ở front-end.
- **Phải nối hai đầu lại với nhau.** Địa chỉ front-end phải được khai báo vào danh sách trắng CORS của máy chủ. Thiếu bước này, trình duyệt chặn mọi lời gọi trong khi máy chủ không ghi nhận lỗi nào — rất khó lần ra nguyên nhân.

Ngoài bản trên Internet, hệ thống còn giữ **một phương án dự phòng chạy hoàn toàn cục bộ**: tệp `docker-compose.full.yml` dựng đủ ba thành phần bằng Docker trên chính máy trình bày. Nếu mạng tại hội đồng chập chờn, chỉ cần một lệnh là có bản chạy đầy đủ ở `localhost` mà không cần Internet — trừ các tính năng AI vốn phải gọi dịch vụ ngoài.

5.8. Danh mục nghiệm thu trước khi bảo vệ

Bảng 5.6. Danh mục kiểm tra trước buổi bảo vệ

Nhóm	Hạng mục kiểm tra
Hạ tầng	/health trả về trạng thái tốt · front-end tải được, không lỗi CORS · migration đã áp dụng và dữ liệu mẫu đã nạp · không biến môi trường nào còn trỏ về máy cục bộ · cơ sở dữ liệu Free trên Render chưa tới ngày hết hạn ~30 ngày kể từ lúc tạo — quá hạn sẽ bị xoá vĩnh viễn, phải nâng cấp hoặc tạo lại trước ngày bảo vệ
Bảo mật	Lịch sử git không chứa tệp <code>.env</code> · không còn lỗ hổng mức cao · header phòng vệ đã bật · giới hạn tần suất có ở <code>/auth</code> và <code>/ai</code> · không phản hồi nào chứa mật khẩu hay mã làm mới phiên

Nhóm	Hạng mục kiểm tra
Chất lượng mã	Biên dịch sạch lỗi ở cả hai dự án · 344/344 phép kiểm đạt · ESLint không còn cảnh báo · không còn lệnh in ra màn hình sót lại · mọi tuyến bất đồng bộ đều bắt lỗi và chuyển về bộ xử lý lỗi
Chống sự cố khi trình diễn	Gọi /health trước 30 phút để đánh thức dịch vụ · chuẩn bị ảnh chụp kết quả AI phòng khi hết hạn mức · mở sẵn tab Mạng để chứng minh không rò rỉ đáp án

5.9. Đánh giá kết quả

Đối chiếu với mục tiêu đặt ra ở mục 1.2, toàn bộ hạng mục bắt buộc của đề tài đã hoàn thành: đủ chức năng của ba vai trò, ba tính năng AI, đóng gói bằng Docker, tích hợp liên tục và cấu hình triển khai lên môi trường thật. Bộ kiểm thử tự động đạt 344/344 và bao phủ toàn bộ các quy tắc nghiệp vụ then chốt, gồm cả bộ hồi quy bổ sung sau đợt review tiền bảo vệ (khóa lộ trình học, vòng đời duyệt nội dung, toàn vẹn dữ liệu khi xóa/sửa đồng thời).

Ba hạn chế cần nêu trung thực:

- **Chưa đo hiệu năng dưới tải thật.** Kết luận về việc tránh được vấn đề N+1 dựa trên phân tích số lượng truy vấn, chưa có số đo thực nghiệm với dữ liệu lớn.
- **Độ bao phủ kiểm thử chưa được đo bằng công cụ.** Bộ kiểm thử được thiết kế theo rủi ro nghiệp vụ chứ không nhằm tới một tỉ lệ bao phủ dòng lệnh cụ thể.
- **Chất lượng nội dung do AI sinh chưa được đánh giá định lượng.** Hệ thống bảo đảm được *cấu trúc* của câu hỏi là hợp lệ, nhưng việc câu hỏi có hay và đúng trọng tâm bài học hay không vẫn phụ thuộc vào đánh giá của giảng viên.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đồ án đã xây dựng hoàn chỉnh nền tảng học tập và kiểm tra trực tuyến LearnQuiz, đáp ứng đầy đủ yêu cầu của Đề tài số 4 cho cả ba vai trò, có tích hợp trợ lý AI, được kiểm thử tự động và có quy trình triển khai lặp lại được.

Nếu phải chọn ba điểm có giá trị kỹ thuật cao nhất trong sản phẩm, đó là:

- **Ranh giới tin cậy được vạch rõ và được kiểm chứng bằng máy.** Mọi quyết định về quyền hạn và điểm số đều nằm ở máy chủ; khẳng định “đáp án đúng không rời khỏi máy chủ” không dừng ở lời cam kết mà được một phép kiểm thử ở mức HTTP chứng minh sau mỗi lần thay đổi mã nguồn.
- **Ràng buộc toàn vẹn được đặt ở đúng tầng.** Chín ràng buộc duy nhất nằm ở tầng cơ sở dữ liệu thay vì ở các câu lệnh điều kiện, nên vẫn đúng cả khi có nhiều yêu cầu đồng thời.
- **Quy tắc nghiệp vụ được tách thành hàm thuần.** Ba mô-đun luật chấm điểm, luật mở khóa và máy trạng thái không phụ thuộc hạ tầng, nhờ đó kiểm thử được tức thì — và thực tế đã giúp phát hiện một lỗi lô-gic trước khi có người dùng nào gặp phải.

6.2. Những điều rút ra được

Kiểm chứng ở trình duyệt là trải nghiệm, kiểm chứng ở máy chủ mới là bảo mật. Mọi thứ chạy trong trình duyệt đều nằm trong tầm kiểm soát của người dùng; ấn một nút bấm không ngăn được ai gọi thẳng API.

Ràng buộc đặt càng gần dữ liệu càng khó bị phá vỡ. Cùng một quy tắc, đặt trong câu lệnh điều kiện thì có thể bị vượt qua khi hai yêu cầu đến đồng thời, đặt ở cơ sở dữ liệu thì không.

Kiểm thử tự động không chỉ để bắt lỗi, mà còn để giữ cho khẳng định luôn đúng theo thời gian. Một phép thử tốt là phép thử vẫn còn giá trị sau khi mã nguồn đã được sửa nhiều lần.

Đầu ra của mô hình ngôn ngữ phải được đối xử như dữ liệu do người lạ gửi lên. Cho AI quyền ghi thẳng vào cơ sở dữ liệu là trao quyền cho một thành phần không xác định.

6.3. Hướng phát triển

Bảng 6.1. Hướng phát triển đề xuất

Hạng mục	Nội dung	Mức độ ưu tiên
Ngân hàng câu hỏi	Tách câu hỏi thành kho dùng chung, sinh đề ngẫu nhiên cho mỗi lượt làm bài — chống học thuộc đáp án	Cao
Chấm điểm có trọng số	Cho phép đặt điểm khác nhau cho từng câu và giới hạn thời gian làm bài	Cao
Chứng chỉ hoàn thành	Sinh tệp PDF chứng nhận khi học viên hoàn thành khóa học và đạt toàn bộ bài kiểm tra	Trung bình
Thông báo	Gửi thư điện tử khi khóa học được duyệt hoặc bị từ chối	Trung bình
Nhật ký thao tác quản trị	Ghi lại ai đã duyệt, từ chối hay khóa tài khoản nào, vào lúc nào	Cao
Đo lường và giám sát	Bổ sung chỉ số vận hành và cảnh báo khi tỉ lệ lỗi tăng đột biến	Trung bình
Kiểm thử giao diện đầu-cuối	Tự động hóa kịch bản người dùng bằng công cụ điều khiển trình duyệt	Trung bình
Cá nhân hóa bằng AI	Gợi ý bài học cần ôn lại dựa trên các câu đã trả lời sai	Thấp

Về mặt kiến trúc, hệ thống hiện tại là một khối duy nhất được phân tầng rõ ràng. Đây là lựa chọn phù hợp với quy mô đề tài. Nếu về sau lưu lượng tăng, hai thành phần có thể tách ra trước tiên là dịch vụ gọi AI (vốn chậm và tốn tài nguyên) và tác vụ tổng hợp thống kê (có thể tính trước theo lịch thay vì tính lúc người dùng yêu cầu).

TÀI LIỆU THAM KHẢO

- [1] Trung Tâm Tin Học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM. *Tài liệu giảng dạy chương trình Lập trình viên Full-stack JavaScript — Khóa 312*, Module 1–4, 2026.
- [2] Trung Tâm Tin Học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM. *Đề tài số 4 — Nền tảng học tập và quiz trực tuyến*, Đề bài đồ án tốt nghiệp, 2026.
- [3] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Luận án tiến sĩ, University of California, Irvine, 2000.
- [4] M. Jones, J. Bradley, N. Sakimura. *RFC 7519 — JSON Web Token (JWT)*. Internet Engineering Task Force, 2015.
- [5] R. Fielding, J. Reschke. *RFC 7231 — Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. IETF, 2014.
- [6] OWASP Foundation. *OWASP Top 10 — 2021: The Ten Most Critical Web Application Security Risks*, 2021.
- [7] OWASP Foundation. *Authentication Cheat Sheet và Mass Assignment Cheat Sheet*, OWASP Cheat Sheet Series.
- [8] Prisma Data, Inc. *Prisma ORM Documentation*. <https://www.prisma.io/docs>
- [9] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
- [10] Meta Open Source. *React Documentation*. <https://react.dev>
- [11] OpenJS Foundation. *Express 4.x API Reference*. <https://expressjs.com/en/4x/api.html>
- [12] Microsoft. *TypeScript Handbook*. <https://www.typescriptlang.org/docs/handbook/>
- [13] Google. *Gemini API Documentation — Structured Output*. <https://ai.google.dev/gemini-api/docs>
- [14] Docker Inc. *Best practices for writing Dockerfiles*. <https://docs.docker.com/build/building/best-practices/>
- [15] K. Schwaber, J. Sutherland. *The Scrum Guide*, 2020.

[16] M. Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

PHỤ LỤC A. BẢNG TRA CỨU MÃ LỖI VÀ THÔNG BÁO

Bảng dưới đây liệt kê các tình huống lỗi nghiệp vụ mà hệ thống sinh ra có chủ đích, kèm mã HTTP tương ứng. Đây là căn cứ để đối chiếu khi kiểm thử và khi trình diễn.

Bảng A.1. Tình huống lỗi nghiệp vụ và mã HTTP

Tình huống	Mã	Thông báo trả về (rút gọn)
Đăng nhập sai địa chỉ thư hoặc sai mật khẩu	401	Email hoặc mật khẩu không đúng
Tài khoản đã bị khóa	403	Tài khoản đã bị khóa
Mã làm mới phiên không khớp bản lưu	401	Phiên đăng nhập không hợp lệ
Ghi danh hai lần cùng một khóa	409	Bạn đã ghi danh khóa học này
Giảng viên ghi danh chính khóa của mình	403	Giảng viên không ghi danh khóa học của mình
Ghi danh khóa chưa được duyệt	403	Khóa học chưa được công khai
Mở bài học chưa tới lượt	403	Cần hoàn thành các bài học trước
Thêm bài học trùng số thứ tự	409	Thứ tự bài học đã tồn tại trong khóa
Gửi duyệt khóa chưa có bài học	400	Khóa học cần có ít nhất một bài học
Duyệt khóa không ở trạng thái chờ	409	Thao tác chỉ thực hiện được khi khóa học ở trạng thái Chờ duyệt
Từ chối mà không nêu lý do	400	Lý do từ chối phải có ít nhất 10 ký tự
Giảng viên tự duyệt khóa của mình	403	Bạn không có quyền thực hiện thao tác này
Sửa đề khi đã có lượt nộp	409	Bài kiểm tra đã có lượt làm bài, không thể thay đề
Hết lượt làm bài	409	Bạn đã dùng hết số lượt làm bài
Xem bài làm của người khác	403	Bạn không có quyền xem bài làm này
Xóa khóa học đã có học viên	409	Khóa học đã có học viên ghi danh
Xóa danh mục còn khóa học	409	Danh mục vẫn còn khóa học
Quản trị viên tự khóa mình	409	Không thể tự khóa tài khoản của chính mình
Khóa quản trị viên đang hoạt động cuối cùng	409	Hệ thống cần ít nhất một quản trị viên đang hoạt động
Chưa cấu hình khóa API của Gemini	503	Tính năng AI chưa sẵn sàng: máy chủ chưa được cấu hình GEMINI_API_KEY
AI trả về dữ liệu sai cấu trúc	422	AI sinh dữ liệu không hợp lệ
Gọi AI quá nhanh	429	Bạn thao tác quá nhanh, vui lòng thử lại sau

Tình huống	Mã	Thông báo trả về (rút gọn)
Gemini không phản hồi trong 20 giây	504	Dịch vụ AI phản hồi quá lâu

PHỤ LỤC B. LƯỢC ĐỒ CƠ SỞ DỮ LIỆU (TRÍCH)

Trích phần khai báo các kiểu liệt kê và ba bảng có ràng buộc đáng chú ý nhất.
Toàn văn nằm trong tệp backend/prisma/schema.prisma.

```
enum Role { student instructor admin }
enum CourseStatus { draft pending published rejected }
enum CourseLevel { beginner intermediate advanced }

model Lesson {
  id Int @id @default(autoincrement())
  courseId Int @map("course_id")
  title String @db.VarChar(200)
  content String?
  videoUrl String? @map("video_url") @db.VarChar(500)
  order Int

  course Course @relation(fields: [courseId], references: [id], onDelete: Cascade)
  quiz Quiz?
  progresses LessonProgress[]

  // Thứ tự bài học là duy nhất trong 1 khóa -> chặn 2 bài cùng order
  @@unique([courseId, order])
  @@map("lessons")
}

model Quiz {
  id Int @id @default(autoincrement())
  lessonId Int @unique @map("lesson_id") // 1-1: mỗi bài học tối đa 1 quiz
  title String @db.VarChar(200)
  passScore Int @default(70) @map("pass_score")
  maxAttempts Int? @map("max_attempts") // null = không giới hạn
  @@map("quizzes")
}

model QuizSubmission {
  id Int @id @default(autoincrement())
  studentId Int @map("student_id")
  quizId Int @map("quiz_id")
  score Int // thang 0-100, do máy chủ tự chấm
  correctCount Int @map("correct_count")
  totalQuestions Int @map("total_questions")
  attemptNo Int @map("attempt_no")

  // Chặn nộp trùng cùng 1 lượt làm bài
  @@unique([studentId, quizId, attemptNo])
  @@map("quiz_submissions")
}
```

PHỤ LỤC C. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY THỬ

C.1. Yêu cầu môi trường

Node.js phiên bản 20 trở lên · npm phiên bản 10 trở lên · Docker Desktop (hoặc PostgreSQL 16 cài sẵn) · Git.

C.2. Ba bước khởi động

```
# 1) Cơ sở dữ liệu
docker compose up -d                # PostgreSQL cổng 5433, Adminer 8080

# 2) Máy chủ
cd backend
copy .env.example .env              # macOS/Linux: cp .env.example .env
npm install
npx prisma migrate dev --name init
npm run seed
npm run dev                          # http://localhost:3000

# 3) Giao diện
cd ../frontend
copy .env.example .env
npm install
npm run dev                          # http://localhost:5173
```

Kiểm tra nhanh: mở <http://localhost:3000/health>, kết quả phải là `{"status": "ok", "db": "up"}`.

C.3. Tài khoản thử nghiệm

Bảng C.1. Tài khoản mẫu (mật khẩu chung: 123456)

Vai trò	Địa chỉ đăng nhập	Họ tên hiển thị
Quản trị viên	admin@learnquiz.vn	Quản trị hệ thống
Giảng viên	instructor@learnquiz.vn	Trần Minh Giảng
Giảng viên 2	instructor2@learnquiz.vn	Lê Thu Hà
Học viên	student@learnquiz.vn	Nguyễn Văn Học
Học viên 2	student2@learnquiz.vn	Phạm Thị Mai

Dữ liệu mẫu gồm 4 danh mục, 3 khóa học chủ đề lập trình (*JavaScript căn bản cho người mới*, *ReactJS thực chiến*, *PostgreSQL và Prisma ORM từ số 0*), 6 bài học và 5 bài kiểm tra.

C.4. Các lệnh thường dùng

Bảng C.2. Lệnh thường dùng

Lệnh	Tác dụng
<code>npm run dev</code>	Chạy máy chủ, tự khởi động lại khi sửa mã nguồn
<code>npm run build</code>	Biên dịch TypeScript sang JavaScript
<code>npm run typecheck</code>	Kiểm tra kiểu, không xuất tệp
<code>npm test</code>	Chạy 344 phép kiểm thử — không cần cơ sở dữ liệu
<code>npm run seed</code>	Nạp lại dữ liệu mẫu (chạy nhiều lần được)
<code>npx prisma studio</code>	Xem và sửa dữ liệu bằng giao diện web
<code>docker compose -f docker-compose.full.yml up --build</code>	Chạy trọn bộ ba thành phần bằng Docker

PHỤ LỤC D. KỊCH BẢN TRÌNH DIỄN KHI BẢO VỆ

Kịch bản dưới đây được thiết kế cho 8 phút, đi qua đủ ba vai trò và cả ba tính năng AI, kết thúc bằng hai bằng chứng kỹ thuật.

Bảng D.1. Kịch bản trình diễn 8 phút

Phút	Vai trò	Thao tác	Điểm cần nhấn
0–1	Khách	Mở trang chủ, duyệt danh sách khóa học, lọc theo danh mục và độ khó	Chỉ khóa học đã duyệt mới hiển thị
1–2	Học viên	Ghi danh khóa <i>JavaScript căn bản</i> , học bài 1, đánh dấu hoàn thành	Tiến độ tăng, bài 2 được mở khóa
2–4	Học viên	Làm quiz, cố tình chọn sai một câu, nộp bài, xem lại đáp án	Máy chủ chấm điểm, không phải trình duyệt
4–5	Học viên	Bấm <i>Vì sao sai?</i> để Gemini giải thích, rồi bấm lại lần nữa	Lần hai lấy từ bản đã lưu, không gọi lại AI
5–6	Giảng viên	Dán nội dung bài học, để Gemini sinh 5 câu hỏi, sửa một câu rồi lưu	AI chỉ tạo bản nháp, người quyết định cuối cùng
6–7	Quản trị viên	Duyệt khóa <i>PostgreSQL</i> và <i>Prisma ORM</i> đang chờ	Khóa học xuất hiện công khai ngay lập tức
7–8	Kỹ thuật	Mở tab Mạng xem phản hồi lấy đề quiz; mở <code>/health</code>	Không có trường <code>isCorrect</code> ; hệ thống quan sát được

— HẾT —